Claude 認證架構師 — 基礎認證

學習指南(依據官方考試指南)

簡介

Claude 認證架構師 — 基礎認證確認專家在實作真實世界中以 Claude 為基礎的解決方案時,能做出妥善的權衡取捨決策。本考試評量對 Claude Code、Claude Agent SDK、Claude API 以及模型上下文協定(MCP)的基礎知識——這些是以 Claude 建構正式環境應用程式的核心技術。

考題以實際的產業情境為基礎:為客戶支援建構代理式系統、設計多代理研究管線、將 Claude Code 整合至 CI/CD、打造開發者生產力工具,以及從非結構化文件中擷取結構化資料。

目標應試者

理想的應試者是以 Claude 設計並交付正式環境應用程式的**解決方案架構師**。你應具備至少 6 個月以下領域的實作經驗:

- **Claude Agent SDK** — 多代理編排、委派給子代理、工具整合、生命週期 Hooks
- **Claude Code** — CLAUDE.md、MCP 伺服器、Agent Skills、規劃模式
- **模型上下文協定(MCP)** — 用於後端整合的工具與資源
- **提示工程** — JSON 結構描述、少樣本範例、資料擷取範本
- **上下文視窗** — 處理長文件、多代理上下文傳遞
- **CI/CD 管線** — 自動化程式碼審查、測試產生
- **升級與可靠性** — 錯誤處理、人在迴路(human-in-the-loop)

考試形式

參數	數值
題型	單選題(四選一)
計分	100–1000 級距,及格分數 **720**
猜題扣分	無(每一題都要作答!)
情境	8 個可能情境中選出 4 個(隨機選取)

考試內容:5 個涵蓋領域

涵蓋領域	權重
1. 代理架構與編排	**27%**
2. 工具設計與 MCP 整合	**18%**
3. Claude Code 設定與工作流程	**20%**
4. 提示工程與結構化輸出	**20%**
5. 上下文管理與可靠性	**15%**

考試情境

情境 1:客戶支援代理

你使用 Claude Agent SDK 建構一個代理,處理退貨、帳務爭議與帳戶問題。該代理使用 MCP 工具(`get_customer`、`lookup_order`、`process_refund`、`escalate_to_human`)。目標是達到 80% 以上的首次接觸解決率,並在適當時機升級。

情境 2:使用 Claude Code 產生程式碼

你使用 Claude Code 加速開發:程式碼產生、重構、除錯、文件撰寫。你需要將它與自訂斜線命令及 CLAUDE.md 設定整合,並了解何時使用規劃模式。

情境 3:多代理研究系統

一個協調者代理將任務委派給專門的子代理:網路研究、文件分析、綜整,以及報告產生。該系統必須產出附帶引用的完整報告。

情境 4:開發者生產力工具

該代理協助工程師探索不熟悉的程式碼庫、產生樣板程式碼,以及自動化例行任務。會使用內建工具(Read、Write、Bash、Grep、Glob)與 MCP 伺服器。

情境 5:用於持續整合的 Claude Code

將 Claude Code 整合至 CI/CD 管線,以進行自動化程式碼審查、測試產生與 pull request 回饋。提示必須經過設計以盡量減少誤報。

情境 6:結構化資料擷取

該系統從非結構化文件中擷取資訊,以 JSON 結構描述驗證輸出,並維持高準確度。它必須正確處理邊界案例。

情境 7:對話式 AI 架構模式

你設計多輪對話式系統,涵蓋上下文視窗管理、跨輪次的指令持續性、記憶策略、用於安全執行的工具設計,以及處理模糊或衝突的使用者輸入。

情境 8:代理式 AI 工具 *(內容缺漏 — 歡迎協助我們補齊!)*

此情境已由應試者回報,但本指南尚未涵蓋。若你在實際考試中遇過此情境的考題,請至 GitHub Issues 分享,讓我們能加入完整內容。你的貢獻將幫助所有準備考試的人。

官方文件

資源	URL
Claude API — Messages	https://platform.claude.com/docs/en/api/messages
Claude API — Tool Use	https://platform.claude.com/docs/en/build-with-claude/tool-use
Claude API — Message Batches	https://platform.claude.com/docs/en/build-with-claude/message-batches
Claude Agent SDK — Overview	https://platform.claude.com/docs/en/agent-sdk/overview
Claude Agent SDK — Hooks	https://platform.claude.com/docs/en/agent-sdk/hooks
Claude Agent SDK — Subagents	https://platform.claude.com/docs/en/agent-sdk/subagents
Claude Agent SDK — Sessions	https://platform.claude.com/docs/en/agent-sdk/sessions
Model Context Protocol (MCP)	https://modelcontextprotocol.io/
MCP — Tools	https://modelcontextprotocol.io/docs/concepts/tools
MCP — Resources	https://modelcontextprotocol.io/docs/concepts/resources
MCP — Servers	https://modelcontextprotocol.io/docs/concepts/servers
Claude Code — Documentation	https://code.claude.com/docs/en/overview
Claude Code — CLAUDE.md and Memory	https://code.claude.com/docs/en/memory
Claude Code — Skills (incl. slash commands)	https://code.claude.com/docs/en/skills
Claude Code — Hooks	https://code.claude.com/docs/en/hooks
Claude Code — Sub-agents	https://code.claude.com/docs/en/sub-agents
Claude Code — MCP Integration	https://code.claude.com/docs/en/mcp
Claude Code — GitHub Actions CI/CD	https://code.claude.com/docs/en/github-actions

Claude Code — GitLab CI/CD	https://code.claude.com/docs/en/gitlab-ci-cd
Claude Code — Headless (non-interactive mode)	https://code.claude.com/docs/en/headless
Prompt Engineering Guide	https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview
Extended Thinking	https://platform.claude.com/docs/en/build-with-claude/extended-thinking
Anthropic Cookbook (code examples)	https://github.com/anthropics/antropic-cookbook

---

第一部分:理論基礎

本部分涵蓋你成功通過考試所需的所有理論。教材是依技術與概念來組織,而非依考試涵蓋領域編排——這有助於你對每個主題建立更深入的理解。

第 1 章:Claude API — 模型互動基礎

> 文件:Messages API | Prompt Engineering

1.1 API 請求結構

Claude API 遵循請求—回應模型。每個送往 Claude Messages API 的請求都包含:

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024,
  "system": "You are a helpful assistant.",
  "messages": [
    {"role": "user", "content": "Hi!"},
    {"role": "assistant", "content": "Hello!"},
    {"role": "user", "content": "How are you?"}
  ],
  "tools": [...],
  "tool_choice": {"type": "auto"}
}
```

主要欄位:

- `model` — 模型選擇(`claude-opus-4-6`、`claude-sonnet-4-6`、`claude-haiku-4-5`)
- `max_tokens` — 回應中的最大 token 數
- `system` — 系統提示(定義模型行為)

- `messages` — 對話歷史(**你必須送出完整的歷史**以維持連貫性)
- `tools` — 可用工具的定義
- `tool_choice` — 工具選擇策略

1.2 訊息角色

`messages` 陣列使用兩種對話角色,再加上一種指示性角色:

- `user` — 使用者訊息,包括工具結果(以 `user` 角色訊息中的 `tool_result` 內容區塊形式送出,而不是獨立的 `tool` 角色)
- `assistant` — 模型回應(送出歷史時會包含),包括工具使用請求(`tool_use` 內容區塊)
- `system` — 可以透過頂層的 `system` 欄位設定(從第一輪就生效),也可以在 `messages` 中以 `{"role": "system", ...}` 內嵌方式加入(從該點起生效,須遵守特定的放置規則 — 見下文)

工具結果不是以 `"tool"` 角色的訊息送出。它們是以 `user` 角色的訊息送出,其內容包含一個 `tool_result` 內容區塊:

```
{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "tool_use_id": "toolu_01...",
      "content": "..."
    }
  ]
}
```

`system` 也可以直接以角色的形式出現在 `messages` 陣列中,而不僅僅是透過頂層的 `system` 參數。這是為了在不使頂層 `system` 欄位的快取前綴失效的情況下,在對話中途加入指示。它有特定的放置規則:

- 必須緊接在一個 `user` 輪次(包括含有 `tool_result` 區塊的輪次)之後,或緊接在以伺服器端工具使用結尾的 `assistant` 輪次之後。
- 必須位於某個 `assistant` 輪次之前,或作為陣列的最後一個元素。
- 不能位於 `tool_use` 區塊與其對應的 `tool_result` 之間 — 這樣做會傳回 400 錯誤。
- 較晚出現的 `system` 訊息(包括對話中途插入的)在後續輪次中,優先於較早的訊息以及頂層的 `system` 欄位。

至關重要:每個 API 請求你都必須送出完整的對話歷史。模型不會在請求之間保留狀態——每次呼叫都是獨立的。

1.3 回應中的 `stop_reason` 欄位

Claude API 的回應包含 `stop_reason`,用以表示模型為何停止生成:

值	說明	動作

`"end_turn"`	模型已完成其回應	將結果呈現給使用者
`"tool_use"`	模型想要呼叫工具	執行該工具並回傳結果
`"max_tokens"`	已達 token 上限	回應被截斷;你可能需要提高上限
`"stop_sequence"`	遇到停止序列	依你的應用程式邏輯處理

對代理式系統而言,`"tool_use"` 和 `"end_turn"` 最為重要——它們控制著代理迴圈。

1.4 系統提示

系統提示是一種特殊指令,用以定義上下文與行為規則。它:

- 不屬於 `messages` 陣列;它會在 `system` 欄位中單獨傳遞
- 優先於使用者訊息
- 載入一次後便在整段對話中持續套用
- 用來定義角色、限制條件與輸出格式

考試重點:系統提示的措辭可能會造成非預期的工具關聯。例如,像「務必驗證客戶」這樣的指令,可能會導致模型過度使用 `get_customer`,即使在不必要的情況下也是如此。

1.5 上下文視窗

上下文視窗是指模型一次能處理的文字總量(以 token 計)。它包含:

- 系統提示
- 完整的訊息歷史
- 工具定義
- 工具結果

上下文視窗的主要問題:

1. **迷失在中間效應:**模型能可靠地處理長輸入開頭與結尾的資訊,但可能會漏掉中間的細節。緩解方式:將關鍵資訊放在靠近開頭或結尾處。
2. **工具結果的累積:**每次工具呼叫都會把輸出加進上下文。如果一個工具回傳 40 個以上的欄位,但只有 5 個有用,那麼大部分的上下文都被浪費了。
3. **漸進式摘要:**在壓縮歷史時,數值、百分比與日期往往會遺失並變得模糊(「大約」、「差不多」、「幾個」)。

第 2 章:工具與 `tool_use`

> 文件:Tool Use

2.1 什麼是 `tool_use`

`tool_use` 是一種讓 Claude 能呼叫外部函式的機制。模型不會直接執行程式碼——它會生成一個結構化的工具呼叫請求;由你的程式碼執行它並回傳結果。

2.2 工具定義

每個工具都使用 JSON 結構描述來定義:

```
{
  "name": "get_customer",
  "description": "Finds a customer by email or ID. Returns the customer profile,
including name, email, order history, and account status. Use this tool BEFORE
lookup_order to verify the customer's identity. Accepts an email (format:
user@domain.com) or a numeric customer_id.",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "Customer email"},
      "customer_id": {"type": "integer", "description": "Numeric customer ID"}
    },
    "required": []
  }
}
```

工具描述至關重要的面向:

1. **描述是主要的選擇機制。** LLM 會根據工具的描述來選擇工具。過於精簡的描述(「Retrieves customer information」)會在工具功能重疊時導致錯誤。
2. **描述中應包含:**
   - 工具的功能與回傳內容
   - 輸入格式與範例值
   - 邊界案例與限制
   - 何時應使用此工具,而非類似的替代工具
3. **避免**在不同工具之間使用相同或重疊的描述。如果 `analyze_content` 與 `analyze_document` 的描述幾乎相同,模型會將兩者混淆。
4. **內建工具 vs MCP 工具:** 代理可能會偏好使用內建工具(Read、Grep),而非功能類似的 MCP 工具。為了避免這種情況,請強化 MCP 工具的描述——突顯內建工具無法提供的具體優勢、獨特資料或上下文。

2.3 `tool_choice` 參數

`tool_choice` 控制模型如何選擇工具:

值	行為	何時使用

`{"type": "auto"}`	模型自行決定要呼叫工具或以文字回答	多數情況的預設值
`{"type": "any"}`	模型**必須**呼叫某個工具	當你需要保證取得結構化輸出時
`{"type": "tool", "name": "extract_metadata"}`	模型**必須**呼叫特定工具	當你需要強制的第一步 / 執行順序時

重要情境:

- `tool_choice: "any"` + 多個擷取工具 → 模型會挑選最合適的一個,但你仍會取得結構化輸出
- 強制選擇 → 當你必須保證特定的第一個動作時(例如,在豐富化之前先執行 `extract_metadata`)

2.4 用於結構化輸出的 JSON 結構描述

搭配 JSON 結構描述使用 `tool_use` 是從 Claude 取得結構化輸出**最可靠**的方式。它能:

- 保證語法上有效的 JSON(沒有缺漏的大括號,沒有多餘的逗號)
- 強制套用所需的結構(必填欄位都存在)
- **不**保證語意正確性(值仍可能有誤)

結構描述設計——關鍵原則:

```
{
  "type": "object",
  "properties": {
    "category": {
      "type": "string",
      "enum": ["bug", "feature", "docs", "unclear", "other"]
    },
    "category_detail": {
      "type": ["string", "null"],
      "description": "Details if category = 'other' or 'unclear'"
    },
    "severity": {
      "type": "string",
      "enum": ["critical", "high", "medium", "low"]
    },
    "confidence": {
      "type": "number",
      "minimum": 0,
      "maximum": 1
    },
    "optional_field": {
      "type": ["string", "null"],
      "description": "Null if the information was not found in the source"
    }
  },
  "required": ["category", "severity"]
}
```

結構描述設計規則:

1. **必填 vs 選填:** 只有在資訊永遠都會有的情況下,才將欄位標記為必填。必填欄位會在資料缺失時,促使模型捏造值。
2. **可為 null 的欄位:** 對於可能不存在的資訊,使用 `"type": ["string", "null"]`。模型可以回傳 `null`,而不會產生幻覺。
3. **含 `"other"` 的列舉:** 加入 `"other"` + 一個詳細說明字串,以避免遺失落在你預先定義類別之外的資料。
4. **列舉值 `"unclear"`:** 用於模型無法有信心挑選類別的情況——誠實的 `"unclear"` 勝過錯誤的類別。

2.5 語法錯誤 vs 語意錯誤

錯誤類型	範例	緩解方式
語法	無效的 JSON、錯誤的欄位型別	搭配 JSON 結構描述使用 `tool_use`(可消除)
語意	總計加不起來、值放錯欄位、幻覺	驗證檢查、帶回饋的重試、自我修正

第 3 章:Claude Agent SDK — 建構代理式系統

> 文件:Agent SDK | Hooks | Subagents | Sessions

3.1 什麼是代理迴圈

代理迴圈是自主執行任務的核心模式。模型不只是回答問題,而是執行一連串動作:

```
1. 帶著工具向 Claude 發送請求
2. 接收回應
3. 檢查 stop_reason:
   - "tool_use" -> 執行工具,將結果附加到歷史紀錄,回到步驟 1
   - "end_turn" -> 任務完成,將結果顯示給使用者
4. 重複直到完成
```

這是一種模型驅動的做法: Claude 會依據上下文與先前的工具結果,決定接下來要呼叫哪個工具。這與動作順序固定的硬編碼決策樹不同。

反模式(應避免):

- 解析助理文字以偵測是否完成(「Task completed」)
- 將任意的迭代上限(例如 `max_iterations=5`)當作主要的停止條件
- 檢查助理是否產生文字內容,以此作為完成訊號

正確做法:唯一可靠的完成訊號是 `stop_reason == "end_turn"`。

3.2 `AgentDefinition` 設定

`AgentDefinition` 是 Claude Agent SDK 中的代理設定物件:

```
agent = AgentDefinition(
    name="customer_support",
    description="Handles customer requests for returns and order issues",
    system_prompt="You are a customer support agent...",
    allowed_tools=["get_customer", "lookup_order", "process_refund",
"escalate_to_human"],
    # 用於協調者:
    # allowed_tools=["Task", "get_customer", ...]
)
```

關鍵參數:

- `name` / `description` — 代理的識別與描述
- `system_prompt` — 含指令的系統提示
- `allowed_tools` — 允許使用的工具清單(最小權限原則)

3.3 中心輻射式:協調者與子代理

多代理架構通常建構為中心輻射式拓撲:

```
        Coordinator
       /     |      \
  Subagent1  Subagent2  Subagent3
   (search)   (analysis)   (synthesis)
```

協調者負責:

- 將任務分解為子任務
- 決定需要哪些子代理(動態選擇)
- 將工作委派給子代理
- 彙整並驗證結果
- 處理錯誤與重試
- 將結果傳達給使用者

關鍵原則:子代理擁有隔離的上下文。

- 子代理**不會**自動繼承協調者的對話歷史紀錄
- 所有必要的上下文都必須在子代理提示中**明確傳遞**
- 子代理不會在多次呼叫之間共享記憶
- 所有溝通都透過協調者流動(以利可觀測性與錯誤控制)

3.4 用於生成子代理的 `Task` 工具

子代理透過 `Task` 工具生成:

```
# 協調者的 allowedTools 必須包含 "Task"
coordinator_agent = AgentDefinition(
    allowed_tools=["Task", "get_customer"]
)
```

明確的上下文傳遞是必要的:

```
# 不好:子代理沒有任何上下文
Task: "分析這份文件"

# 好:提示中包含完整上下文
Task: "分析以下文件。
文件:[完整文件文字]
先前的搜尋結果:[網路搜尋結果]
輸出格式要求:[schema]"
```

平行生成:協調者可以在單一回應中呼叫多個 `Task`——這些子代理會平行執行:

```
# 一次協調者回應包含:
Task 1: "搜尋關於 X 的文章"
Task 2: "分析文件 Y"
Task 3: "搜尋關於 Z 的文章"
# 三者同時執行
```

3.5 Agent SDK 中的 Hooks

Hooks 允許在代理生命週期的特定節點進行攔截與轉換。

PostToolUse 會在工具結果提供給模型之前攔截它:

```
# 範例:將來自不同 MCP 工具的日期格式正規化
@hook("PostToolUse")
def normalize_dates(tool_result):
    # 將 Unix 時間戳記轉換 -> ISO 8601
    # 將 "Mar 5, 2025" 轉換 -> "2025-03-05"
    return normalized_result
```

對外呼叫攔截 Hook 會阻擋違反政策的動作:

```python
# 範例:阻擋超過 $500 的退款
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
    if tool_call.name == "process_refund" and tool_call.args.amount > 500:
        return redirect_to_escalation(tool_call)
```

關鍵差異:Hooks 與提示指令

屬性	Hooks	提示指令
保證	**確定性**(100%)	**機率性**(>90%,並非 100%)
何時使用	關鍵業務規則、財務操作、合規	一般偏好、建議、格式設定
範例	阻擋退款 > $500	「在升級前先嘗試解決」

規則: 當失敗會造成財務、法律或安全後果時——使用 Hooks,而非提示。

第 4 章:模型上下文協定(MCP)

> 文件:MCP | Tools | Resources | Servers

4.1 什麼是 MCP

模型上下文協定(MCP)是一種用於將外部系統連接至 Claude 的開放協定。MCP 定義了三種主要的資源類型:

1. **Tools** — 代理可以呼叫以執行動作的函式(CRUD 操作、API 呼叫、命令執行)
2. **Resources** — 代理可以讀取以作為上下文的資料(文件、資料庫結構描述、內容目錄)
3. **Prompts** — 用於常見任務的預先定義提示範本

4.2 MCP 伺服器

MCP 伺服器是一個實作 MCP 協定並提供工具/資源的程序。當你連接至 MCP 伺服器時:

- 所有工具都會自動被探索
- 來自所有已連接伺服器的工具會一次全部可用
- 工具描述決定了模型將如何使用它們

4.3 設定 MCP 伺服器

專案設定(`.mcp.json`) — 供團隊使用:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    },
    "jira": {
      "command": "npx",
      "args": ["-y", "mcp-server-jira"],
      "env": {
        "JIRA_TOKEN": "${JIRA_TOKEN}"
      }
    }
  }
}
```

重點:

- `.mcp.json` 儲存於專案根目錄,並由版本控制管理
- 環境變數(`${GITHUB_TOKEN}`)用於存放機密——token 本身不會被提交
- 對所有專案貢獻者皆可用

使用者設定(`~/.claude.json`) — 供個人/實驗性伺服器使用:

- 儲存於使用者的家目錄
- 不透過版本控制共用
- 適合個人實驗與測試

選擇伺服器:

- 對於標準整合(Jira、GitHub、Slack),優先採用現有的社群 MCP 伺服器
- 只有針對獨特、團隊專屬的工作流程才自行建置伺服器

4.4 MCP 中的 `isError` 旗標

當 MCP 工具遇到錯誤時,它會在回應中使用 `isError: true`。這會向代理發出該呼叫失敗的訊號。

結構化錯誤(良好):

```
{
  "isError": true,
  "content": {
    "errorCategory": "transient",
    "isRetryable": true,
    "message": "The service is temporarily unavailable. Timeout while calling the
orders API.",
    "attempted_query": "order_id=12345",
    "partial_results": null
  }
}
```

通用錯誤(反模式):

```
{
  "isError": true,
  "content": "Operation failed"
}
```

通用錯誤無法為代理的決策提供任何資訊——它應該重試、變更查詢,還是升級?

4.5 MCP 資源

資源是代理可以請求的資料,用來取得上下文而不必採取行動:

- 內容目錄(例如所有專案任務的清單、階層式導覽)
- 資料庫結構描述(理解資料結構)
- 文件(API 參考、內部指南)
- 議題／任務摘要

資源的優勢: 代理不需要透過探索性的工具呼叫就能理解有哪些資料存在。資源提供了一份即時的「地圖」。

第 5 章:Claude Code — 設定與工作流程

> 文件:Claude Code | Memory / CLAUDE.md | Skills | MCP | Hooks | Sub-agents | GitHub Actions | Headless

5.1 CLAUDE.md 階層

CLAUDE.md 是 Claude Code 的指令檔。它具有三層階層:

```
1. User-level: ~/.claude/CLAUDE.md
   - 僅套用於該使用者
   - 不透過 VCS 共享
   - 個人偏好與工作風格

2. Project-level: .claude/CLAUDE.md 或根目錄的 CLAUDE.md
   - 套用於所有專案貢獻者
   - 透過 VCS 管理
   - 程式碼規範、測試規範、架構決策

3. Directory-level: 子目錄中的 CLAUDE.md
   - 在處理該目錄中的檔案時套用
   - 該部分程式碼庫特有的慣例
```

常見錯誤: 新進團隊成員沒有收到專案指令,因為這些指令被放在 `~/.claude/CLAUDE.md` (user-level)而非 `.claude/CLAUDE.md` (project-level)。

5.2 `@path` 語法(檔案匯入)

CLAUDE.md 可以使用 `@path` 參照外部檔案,使設定模組化:

```
# Project CLAUDE.md

Coding standards are described in @./standards/coding-style.md
Test requirements are in @./standards/testing-requirements.md
Project overview is in @README.md and dependencies are in @package.json
```

`@path` 的規則:

- 在檔案路徑前緊接著使用 `@` (沒有空格)
- 支援相對路徑與絕對路徑
- 相對路徑會相對於包含該匯入的檔案來解析
- 最大匯入巢狀深度為 5

這可以避免重複,並讓每個套件只包含相關的規範。

5.3 `.claude/rules/` 目錄

`.claude/rules/` 是單一龐大 CLAUDE.md 的替代方案,用於依主題組織規則:

```
.claude/rules/
  testing.md          -- 測試慣例
  api-conventions.md  -- API 慣例
  deployment.md       -- 部署規則
  react-patterns.md   -- React 模式
```

關鍵特性:使用 `paths` 的 YAML frontmatter 進行條件式載入:

```
---
paths: ["src/api/**/*"]
---

For API files, use async/await with explicit error handling.
Each endpoint must return a standard response wrapper.
```

```
---
paths: ["**/*.test.tsx", "**/*.test.ts"]
---

Tests must use describe/it blocks.
Use data factories instead of hardcoding.
Do not mock the database—use a test database.
```

運作方式:

- 只有在 Claude Code 編輯符合 `paths` 模式的檔案時,規則才會被載入
- 這可以節省上下文與 tokens——不相關的規則不會被載入
- Glob 模式讓你能依檔案類型套用慣例,而不受位置影響(對於散落在程式碼庫各處的測試而言相當理想)

何時使用搭配 `paths` 的 `.claude/rules/`,而非 directory-level CLAUDE.md:

- 搭配 `paths` 的 `.claude/rules/` — 當慣例適用於散布在許多目錄中的檔案時(測試、遷移)
- Directory-level CLAUDE.md — 當慣例與特定目錄綁定,且其他地方不需要時

5.4 自訂斜線命令與技能

> **注意:**在目前的 Claude Code 版本中,自訂命令(`.claude/commands/`)已與技能(`.claude/skills/`)整合。兩種格式都會建立 `/name` 命令。考試指南所引用的是 `.claude/commands/`——該格式仍受支援。

斜線命令是可重複使用的提示範本,透過 `/name` 來叫用:

`.claude/commands/` 格式(舊版,仍受支援):

```
.claude/commands/
  review.md        -- /review -- 標準程式碼審查
  test-gen.md      -- /test-gen -- 測試產生
```

`.claude/skills/` 格式(目前):

```
.claude/skills/
  review/SKILL.md  -- /review -- 含 frontmatter 設定
  test-gen/SKILL.md
```

專案命令(`.claude/commands/` 或 `.claude/skills/`):

- 儲存在 VCS 中,且任何人在複製儲存庫時都能取用
- 確保團隊間工作流程一致

使用者命令(`~/.claude/commands/` 或 `~/.claude/skills/`):

- 不透過 VCS 分享的個人命令
- 用於個人的工作流程

5.5 技能 — `.claude/skills/`

技能是透過 SKILL.md frontmatter 設定的進階命令:

```
---
context: fork
allowed-tools: ["Read", "Grep", "Glob"]
argument-hint: "要分析的目錄路徑"
---

分析指定目錄中的程式碼結構。
輸出一份關於相依性與架構模式的報告。
```

Frontmatter 參數:

參數	描述
`context: fork`	在隔離的子代理中執行技能。冗長的輸出不會汙染主工作階段
`allowed-tools`	限制有哪些工具可用(安全性——例如,若未允許,技能就無法刪除檔案)
`argument-hint`	在未帶參數叫用時,提示要求提供一個引數

何時使用技能 vs CLAUDE.md:

- **技能** — 針對特定任務(審查、分析、產生)的按需叫用
- **CLAUDE.md** — 永遠載入的一般標準與慣例

個人技能(`~/.claude/skills/`):

- 以不同名稱建立個人變體,如此就不會影響團隊成員

5.6 規劃模式 vs 直接執行

規劃模式:

- 模型只進行調查與規劃;不會做出變更
- 使用 Read、Grep、Glob 探索程式碼庫
- 產生一份由使用者核准的實作計畫
- 安全探索,不產生副作用

何時使用規劃模式:

- 大型變更(數十個檔案)
- 多種看似可行的做法(微服務:該如何定義邊界?)
- 架構決策(用哪個框架?採用什麼結構?)
- 不熟悉的程式碼庫(變更前必須先理解)
- 影響 45 個以上檔案的函式庫遷移

何時使用直接執行:

- 具有明確堆疊追蹤的單檔修正
- 新增一項驗證檢查
- 充分理解、不模糊的變更

結合做法:

1. 以規劃模式進行調查與設計
2. 使用者核准計畫
3. 以直接執行來實作已核准的計畫

Explore 子代理 — 用於探索程式碼庫的專用子代理:

- 將冗長的輸出與主上下文隔離
- 只回傳一份摘要
- 防止在多階段任務中耗盡上下文視窗

5.7 `/compact` 命令

`/compact` 是用於壓縮上下文的內建命令:

- 摘要先前的歷史記錄,以釋放上下文視窗
- 用於長時間的調查工作階段,當上下文被冗長的工具輸出填滿時
- 風險:確切的數值、日期與特定細節可能在摘要過程中遺失

5.8 `/memory` 命令

`/memory` 是用於管理工作階段間記憶的內建命令:

- 開啟 `CLAUDE.md` 檔案進行編輯,讓你能儲存筆記、偏好設定與上下文
- 資訊會跨工作階段持續保存,並在啟動時自動載入
- 適合用於儲存專案慣例、使用者偏好、常用命令以及目前的工作上下文
- 取代在每個工作階段重新解釋相同指令的做法

5.9 用於 CI/CD 的 Claude Code CLI

`-p` (或 `--print`)旗標:

```
claude -p "Analyze this pull request for security issues"
```

- 非互動模式:處理提示、輸出到 stdout,然後結束
- 不等待使用者輸入
- 在 CI/CD 管線中執行 Claude 的唯一正確方式

用於 CI 的結構化輸出:

```
claude -p "Review this PR" --output-format json --json-schema
'{"type":"object",...}'
```

- `--output-format json` — 以 JSON 格式輸出
- `--json-schema` — 依據結構描述驗證輸出
- 結果可被解析,以自動張貼行內 PR 留言

工作階段上下文隔離: 產生程式碼的同一個 Claude 工作階段,在審查該程式碼時往往較不有效(模型會保留其推理上下文,較不可能挑戰自己的決定)。請使用獨立的執行個體進行審查。

避免重複留言: 在新的提交後重新審查時,將先前的審查結果納入上下文,並指示 Claude 只回報新的或尚未解決的問題。

5.10 `fork_session` 與工作階段管理

`--resume <session-name>` 恢復一個具名工作階段:

```
claude --resume investigation-auth-bug
```

- 以已儲存的上下文繼續先前的對話
- 適合跨多個工作階段的長期調查
- 風險:若檔案自先前工作階段以來已變更,工具結果可能已過時

`fork_session` 從共享上下文建立一個獨立分支:

```
Codebase investigation
         |
    fork_session
    /           \
Approach A:      Approach B:
Redux            Context API
```

- 兩個分叉都繼承到分支點為止的上下文
- 之後,它們各自獨立分歧
- 適合用於比較不同做法或測試策略

何時應改為開始新工作階段而非恢復:

- 工具結果已過時(檔案已變更)
- 已經過了很長時間,上下文已劣化
- 與其用舊的工具資料恢復,不如以「以下是我們發現內容的簡短摘要:……」重新開始

第 6 章:提示工程 — 進階技巧

> 文件:提示工程 | Anthropic Cookbook

6.1 少樣本提示

少樣本提示是在提示中加入 2–4 個輸入／輸出範例,以示範預期的行為。

為何少樣本比文字描述更有效:

- 像「更精確一點」這樣模糊的指令可能有許多種解讀方式
- 範例能毫不含糊地展示預期的格式與決策邏輯
- 模型會將該模式類推到新的案例(它不只是重複那些範例)

少樣本範例的類型及其使用時機:

1. **用於模糊情境的範例:**

```
Request: "My order is broken"
Action: Call get_customer -> lookup_order -> check status.
Rationale: “broken” may mean a damaged item; you need order details.

Request: "Get me a manager"
Action: Immediately call escalate_to_human.
Rationale: The customer explicitly requests a human. Do not attempt to solve
autonomously.
```

2. **用於輸出格式化的範例:**

```
Finding example:
{
  "location": "src/auth/login.ts:42",
  "issue": "SQL injection in the username parameter",
  "severity": "critical",
  "suggested_fix": "Use a parameterized query"
}
```

3. **用於區分可接受與有問題程式碼的範例:**

```
// Acceptable (do not flag):
const items = data.filter(x => x.active);

// Problem (flag):
const items = data.filter(x => x.active == true); // Use strict equality ===
```

4. **用於從不同文件格式擷取的範例:**

```
Document with inline citations:
"As shown in the study (Smith, 2023), the rate is 42%."
-> {"value": "42%", "source": "Smith, 2023", "type": "inline_citation"}

Document with bibliography references:
"The rate is 42%. [1]"
-> {"value": "42%", "source": "reference_1", "type": "bibliography"}
```

5. **用於非正式量測的範例:**

```
Text: "about two handfuls of rice"
-> {"amount": "~100g", "original_text": "two handfuls", "precision": "approximate"}

Text: "a pinch of salt"
-> {"amount": "~1g", "original_text": "a pinch", "precision": "approximate"}
```

少樣本對於擷取非正式且非標準的量測單位特別有效,這類單位過於多樣,難以單純以規則式指令處理。

提示中的格式正規化規則: 當使用嚴格的 JSON 結構描述進行結構化輸出時,在提示中加入正規化規則:

```
Normalization:
- Dates: always ISO 8601 (YYYY-MM-DD); "yesterday" -> compute an absolute date
- Currency: numeric amount + currency code; "five bucks" -> {"amount": 5, "currency":
"USD"}
- Percentages: decimal fraction; "half" -> 0.5
```

這可防止語意錯誤,亦即 JSON 在語法上有效、但數值卻不一致的情況。

6.2 明確的判準 vs 模糊的指令

不佳(模糊):

```
檢查程式碼註解的正確性。
請保守一點—只回報高信心的發現。
```

良好(明確判準):

```
僅在符合下列情況時,才將某則註解標記為有問題:
1. 該註解描述的行為與實際的程式碼行為相互矛盾
2. 該註解引用了不存在的函式或變數
3. TODO/FIXME 註解所指的 bug 已在程式碼中修正完成

請勿標記:
- 僅是風格上過時的註解
- 用字稍有不精確的註解
- 缺漏的註解(那屬於另一個類別)
```

用範例定義嚴重程度判準:

```
CRITICAL: 使用者端的執行階段失敗
  範例: 處理付款時發生 NullPointerException

HIGH: 安全性漏洞
  範例: SQL injection、XSS、缺漏授權檢查

MEDIUM: 無立即影響的邏輯 bug
  範例: 排序錯誤、差一錯誤(off-by-one error)

LOW: 程式碼品質
  範例: 重複程式碼、對小型資料採用次佳演算法
```

6.3 提示鏈接

提示鏈接將一項複雜任務拆解為一連串聚焦的步驟:

```
Step 1: Analyze auth.ts (僅限本檔案內部的問題)
       -> 輸出: auth.ts 的問題清單

Step 2: Analyze database.ts (僅限本檔案內部的問題)
       -> 輸出: database.ts 的問題清單

Step 3: 整合遍歷 (跨檔案相依)
       -> 輸出: 模組邊界處的問題
```

為何這很重要:

- 避免**注意力稀釋**——當模型一次收到太多檔案時,可能會漏掉某些檔案中的 bug,同時對其他檔案只提供淺薄的評論
- 確保每個檔案的分析品質一致
- 允許對跨檔案的互動進行獨立分析

何時使用提示鏈接 vs 動態分解:

- **提示鏈接** — 可預測、可重複的任務(程式碼審查、檔案遷移)
- **動態分解** — 開放式的調查,子任務只有在執行過程中才會變得明朗

6.4 「訪談」模式

在實作解決方案之前,Claude 會先問釐清問題:

```
Claude: "在為這個 API 實作快取之前,先問幾個問題:
1. 你偏好哪一種快取失效策略—TTL 還是事件驅動?
2. 當快取無法使用時,陳舊資料是否可以接受?
3. 快取應該是依使用者(per-user)還是全域(global)?
4. 預期要快取的資料量有多大?"
```

何時這很有用:

- 不熟悉的領域(金融科技、醫療保健、法律系統)
- 帶有不明顯影響的任務(快取策略、失敗模式)
- 存在多種可行方法,而最佳選擇取決於上下文

6.5 驗證與帶回饋的重試

當擷取出的資料未通過驗證時:

```
Step 1: 從文件中擷取資料
Step 2: 驗證 (Pydantic、JSON Schema、業務規則)
Step 3: 若有錯誤—帶著上下文重試:
  - 原始文件
  - 先前(錯誤的)擷取結果
  - 具體的錯誤: "Field 'total' = 150, but sum(line_items) = 145. Re-check values."
```

何時重試會有效:

- 格式錯誤(日期格式錯誤)
- 結構錯誤(欄位放在錯誤的位置)
- 算術不一致(模型可以重新檢查)

何時重試沒有幫助:

- 該資訊在來源文件中根本不存在
- 所需的上下文在外部(資料位於另一份未提供的文件中)

Pydantic 作為驗證工具: Pydantic 是一個用於以結構描述為基礎進行資料驗證的 Python 函式庫。對於考試而言,重點如下:

- **結構驗證:** 在從 Claude 收到 JSON 之後,於程式碼中檢查型別、必填性、enum 約束
- **語意驗證:** 自訂驗證器強制執行業務邏輯(各項目總和等於 total;start_date < end_date)
- **驗證—重試迴圈:** 當 Pydantic 驗證失敗時,建構一則錯誤訊息,並帶著錯誤上下文重新提示 Claude
- **JSON Schema 產生:** Pydantic 模型可為 `tool_use` 產生 JSON Schema,提供單一可信來源

6.6 自我修正

一種偵測內部矛盾的模式:

```
{
  "stated_total": "$150.00",
  "calculated_total": "$145.00",
  "conflict_detected": true,
  "line_items": [
    {"name": "Widget A", "price": 75.00},
    {"name": "Widget B", "price": 70.00}
  ]
}
```

模型同時擷取陳述值與計算值——若兩者不同,`conflict_detected` 讓你能處理這項差異。

第 7 章:Message Batches API

> 文件:Message Batches

7.1 概觀

Message Batches API 讓你提交多批請求進行非同步處理:

屬性	值
節省成本	相較同步呼叫節省 **50%**
處理時間視窗	最長 **24 小時**(無延遲 SLA 保證)
多輪工具呼叫	**不支援**(一個請求 = 一個回應)
關聯對應	以 `custom_id` 欄位連結請求與回應

7.2 何時使用批次 API 與同步 API

任務	API	原因
合併前 PR 檢查	**同步**	開發者正在等待;24 小時無法接受
隔夜技術債報告	**批次**	結果在早上前需要即可;節省 50%
每週安全稽核	**批次**	不緊急;節省 50%
互動式程式碼審查	**同步**	需要即時回應
處理 10,000 份文件	**批次**	大量處理;節省幅度可觀

7.3 使用 `custom_id`

```
{
  "custom_id": "doc-invoice-2024-001",
  "params": {
    "model": "claude-sonnet-4-6",
    "max_tokens": 1024,
    "messages": [{"role": "user", "content": "從以下內容擷取資料:..."}]
  }
}
```

`custom_id` 讓你能夠:

- 將結果連結回原始文件
- 失敗時,只重新提交失敗的文件
- 避免重新處理已成功的文件

7.4 處理批次中的失敗

1. 提交一批 100 份文件
2. 95 份成功;5 份失敗(超出上下文限制)
3. 以 `custom_id` 找出失敗項
4. 修改策略(例如,將長文件切分成多個區塊)
5. 只重新提交 5 份失敗的文件

7.5 SLA 規劃

若你需要在 30 小時內取得結果,而批次 API 最長可能耗時 24 小時:

- 提交時間視窗:30 - 24 = **6 小時**
- 批次最晚必須在期限前 24 小時提交
- 若需頻繁提交,切分成 4 小時的時間視窗

第 8 章:任務分解策略

8.1 固定管線(提示鏈接)

每個步驟都事先定義:

```
文件 -> 中繼資料擷取 -> 資料擷取 -> 驗證 -> 加值擴充 -> 最終輸出
```

何時使用:

- 任務結構可預測(審查總是遵循相同範本)
- 所有步驟事先已知
- 你需要穩定性與可重現性

8.2 動態自適應分解

子任務會根據中間結果產生:

```
1. "Add tests for a legacy codebase"
2. -> 首先:盤點結構(Glob、Grep)
3. -> 發現:3 個模組沒有測試,2 個僅有部分覆蓋
4. -> 排定優先順序:從付款模組開始(高風險)
5. -> 進行中:發現對某個外部 API 的相依性
6. -> 調整:在撰寫測試前先為該外部 API 加入 mock
```

何時使用:

- 開放式的調查型任務
- 當完整範圍在一開始未知時
- 當每一步都取決於前一步的結果時

8.3 多遍程式碼審查

對於包含 10 個以上檔案的 pull request:

```
Pass 1(逐檔):分析 auth.ts -> 列出局部問題
Pass 1(逐檔):分析 database.ts -> 列出局部問題
Pass 1(逐檔):分析 routes.ts -> 列出局部問題
...
Pass 2(整合):分析檔案之間的關聯
  -> 跨檔問題:型別不一致、循環相依
```

為什麼一次掃過 14 個檔案不好:

- 注意力稀釋:對某些檔案做了深入分析,對其他檔案則流於表面
- 註解不一致:某個模式在一個檔案被標記,卻在另一個檔案被放行
- 漏掉錯誤:明顯的錯誤因認知過載而被略過

第 9 章:升級與人在迴路(Human-in-the-Loop)

9.1 何時升級至真人

升級觸發條件(明確規則):

情境	動作

客戶明確要求「幫我找主管」	立即升級;不要嘗試自行解決
政策未涵蓋該請求	升級(例如政策未規範時的競品比價折扣)
代理無法取得進展	在嘗試合理次數後升級
超過門檻的金融操作	升級(最好透過 hook 強制執行,而非靠提示)
搜尋客戶時出現多筆相符結果	要求提供額外的識別資訊;不要猜測

哪些不是可靠的觸發條件:

不可靠的方法	為何會失效
情緒分析	客戶情緒與案件複雜度並不相關
模型自評信心(1–10)	模型可能自信地犯錯;校準很差
自動分類器	過度設計;可能需要你沒有的訓練資料

9.2 升級模式

立即升級:

```
Customer: "I want to speak to a manager"
Agent: [立即呼叫 escalate_to_human]
NOT: "I can help with your issue, let me..."
```

嘗試解決後再升級:

```
Customer: "My refrigerator broke two days after purchase"
Agent: [查看訂單,提供保固更換]
若客戶不滿意 -> 升級
```

細緻的升級(同理 → 解決 → 客戶重申時升級):

```
Customer: "This is outrageous, I'm very unhappy with the quality!"
Agent: [同理對方的不滿] "I understand your frustration."
       [提供解決方案] "I can offer a replacement or a refund."
Customer: "No, I want to talk to someone!"
Agent: [客戶再次堅持 -> 立即升級]
```

關鍵原則:先同理對方的情緒,接著提出具體的解決方案,只有在客戶重申想找真人時才升級。不要在客戶第一次表達不滿時就升級(那與要求找主管並不相同)。

政策缺口的升級:

```
Customer: "Competitor X has this item 30% cheaper—give me a discount"
Policy: 僅涵蓋自家網站上的價格調整
Agent: [升級 — 政策未涵蓋競品比價]
```

9.3 結構化交接協定

升級時,代理應將結構化摘要傳遞給真人:

```
{
  "customer_id": "CUST-12345",
  "customer_name": "Ivan Petrov",
  "issue_summary": "Refund request for a damaged item",
  "order_id": "ORD-67890",
  "root_cause": "Item arrived damaged; photos attached",
  "actions_taken": [
    "Verified customer via get_customer",
    "Confirmed order via lookup_order",
    "Offered a standard replacement — customer insists on a refund"
  ],
  "refund_amount": "$89.99",
  "recommended_action": "Approve a full refund",
  "escalation_reason": "Customer requested to speak with a manager"
}
```

真人操作員無法存取完整的對話逐字記錄——他們只看得到這份摘要。因此它必須完整且自成一體。

9.4 信心校準與人工監督

對於資料擷取系統:

1. **欄位層級信心分數:** 模型為每個擷取出的欄位輸出一個信心分數
2. **校準:** 使用已標註的驗證集來調整門檻值
3. **路由:**
   - 高信心 + 穩定準確度 -> 自動化處理
   - 低信心或來源不明確 -> 人工審查

分層隨機抽樣:

- 即使是高信心的擷取結果,也要定期稽核一份樣本
- 整體 97% 的準確度可能掩蓋了某種特定文件類型 40% 的錯誤
- 依文件類型與欄位分析準確度,而不只是看整體

第 10 章:多代理系統中的錯誤處理

10.1 錯誤類別

類別	範例	可重試	代理動作
Transient	Timeout、503、網路故障	是	以指數退避重試

Validation	輸入格式無效、缺少必填欄位	否(修正輸入)	修改請求後重試
Business	政策違規、超出門檻	否	向使用者說明;提出替代方案
Permission	存取被拒	否	升級

10.2 錯誤處理反模式

反模式	問題	正確做法
通用狀態「search unavailable」	協調者無法判斷如何復原	回傳錯誤類型、查詢、部分結果、替代方案
靜默抑制(空結果 = 成功)	協調者以為沒有符合項目,但其實是失敗	區分「沒有結果」與「搜尋失敗」
因單一失敗而中止整個工作流程	你會失去所有部分結果	以部分結果繼續;標註缺口
在子代理內部無限重試	延遲與資源浪費	本地復原(1–2 次重試),然後傳播給協調者

10.3 結構化的子代理錯誤

```
{
  "status": "partial_failure",
  "failure_type": "timeout",
  "attempted_query": "AI impact on music industry 2024",
  "partial_results": [
    {"title": "AI Music Generation Report", "url": "...", "relevance": 0.8}
  ],
  "alternative_approaches": [
    "Try a narrower query: 'AI music composition tools'",
    "Use an alternative data source"
  ],
  "coverage_impact": "Not covered: AI impact on music production"
}
```

這為協調者提供了做出決策所需的資訊:

- 以修改後的查詢重試?
- 使用部分結果?
- 委派給不同的子代理?
- 不含本節繼續進行,並標註缺口?

10.4 最終綜整中的涵蓋範圍標註

```
## Report: AI Impact on Creative Industries

### Visual Art (FULL COVERAGE)
[research results]

### Music (PARTIAL COVERAGE — search agent timeout)
[partial results]
⚠️ Note: coverage for this section is limited due to a timeout in the search agent.

### Literature (FULL COVERAGE)
[research results]
```

第 11 章:正式環境系統中的上下文管理

11.1 將事實擷取至獨立區塊

與其依賴對話歷史(在摘要過程中會劣化),不如將關鍵事實擷取至一個結構化區塊中:

```
=== CASE FACTS (updated whenever a new fact appears) ===
Customer ID: CUST-12345
Order ID: ORD-67890
Order Date: 2025-01-15
Order Amount: $89.99
Issue: Damaged item on delivery
Customer Request: Full refund
Status: Pending manager approval
===
```

無論歷史如何被摘要,都要在每個提示中納入這個區塊。

11.2 修剪工具結果

如果 `lookup_order` 回傳了 40 個以上的欄位,但目前任務只需要其中 5 個:

```
# PostToolUse hook:只保留相關的欄位
@hook("PostToolUse", tool="lookup_order")
def trim_order_fields(result):
    return {
        "order_id": result["order_id"],
        "status": result["status"],
        "total": result["total"],
        "items": result["items"],
        "return_eligible": result["return_eligible"]
    }
```

這能節省上下文並減少雜訊。

11.3 位置感知輸入

放置關鍵資訊時,要將迷失在中間效應納入考量:

```
[KEY FINDINGS — at the top]
Found 3 critical vulnerabilities...

[DETAILED RESULTS — middle]
=== File auth.ts ===
...
=== File database.ts ===
...

[ACTION ITEMS — at the end]
Priority: fix auth.ts vulnerabilities before merge.
```

11.4 暫存檔

在長時間的調查中,代理可以將關鍵發現寫入暫存檔(scratchpad):

```
# investigation-scratchpad.md
## Key findings
- PaymentProcessor in src/payments/processor.ts inherits from BaseProcessor
- refund() is called from 3 places: OrderController, AdminPanel, CronJob
- External PaymentGateway API has a rate limit of 100 req/min
- Migration #47 added refund_reason (NOT NULL) — 2024-12-01
```

當上下文劣化時(或在新的工作階段中),代理可以查詢暫存檔,而不必重新執行探索。

11.5 委派給子代理以保護上下文

```
Main agent: "Investigate dependencies of the payments module"
  -> Subagent (Explore): reads 15 files, traces imports
  -> Returns: "Payments depends on AuthService, OrderModel, and the external
PaymentGateway API"

Main agent: keeps one line in context instead of 15 files
```

獨立的上下文層: 在多代理系統中,每個子代理都在有限的上下文預算內運作——它只接收其任務所需的資訊。協調者扮演一個獨立的上下文層:它彙整子代理的輸出、儲存全域狀態,並配置上下文。這能防止「上下文洩漏」,亦即某個代理用與其他代理無關的資訊耗盡了視窗。

子代理受限的上下文預算:

- 傳送最少的上下文:一個明確的任務 + 必要的資料
- 指示子代理回傳結構化結果,而非原始資料傾印
- 使用 `allowedTools` 來限制子代理的工具集——工具越少代表干擾越少、上下文成本越低

11.6 結構化狀態持久化(用於當機復原)

每個代理都會將其狀態匯出至一個已知位置:

```
// agent-state/web-search-agent.json
{
  "status": "completed",
  "queries_executed": ["AI music 2024", "AI music composition"],
  "results_count": 12,
  "key_findings": [...],
  "coverage": ["music composition", "music production"],
  "gaps": ["music distribution", "music licensing"]
}
```

協調者在恢復時會載入一份清單(manifest):

```
// agent-state/manifest.json
{
  "web-search": "completed",
  "doc-analysis": "in_progress",
  "synthesis": "not_started"
}
```

第 12 章:保留出處來源

12.1 出處遺失問題

在彙整多個來源的結果時,「主張 → 來源」的連結可能會遺失:

```
不佳:「AI 音樂市場估值為 32 億美元。」(無來源、無年份。)

良好:
{
  "claim": "The AI music market is estimated at $3.2B.",
  "source_url": "https://example.com/report",
  "source_name": "Global AI Music Report 2024",
  "publication_date": "2024-06-15",
  "confidence": 0.9
}
```

12.2 處理衝突資料

當兩個來源提供不同的數值時:

```
{
  "claim": "Share of AI-generated music on streaming platforms",
  "values": [
    {
      "value": "12%",
      "source": "Spotify Annual Report 2024",
      "date": "2024-03",
      "methodology": "Automated classification"
    },
    {
      "value": "8%",
      "source": "Music Industry Association Survey",
      "date": "2024-07",
      "methodology": "Survey of 500 labels"
    }
  ],
  "conflict_detected": true,
  "possible_explanation": "Difference in methodology and time period"
}
```

不要任意選擇其中一個數值。連同出處標註一併保留兩者,讓協調者決定。

12.3 納入日期以正確詮釋

若沒有日期,時間上的差異可能被誤解為相互矛盾:

```
不佳:「來源 A 說 10%,來源 B 說 15%。相互矛盾。」
良好:「來源 A(2023)說 10%,來源 B(2024)說 15%。可能是一年內成長了 +5%。」
```

12.4 依內容類型呈現

不要把所有內容硬塞進單一格式:

- 財務資料 -> 表格
- 新聞與分析 -> 散文
- 技術發現 -> 結構化清單
- 時間序列 -> 依時間順序排列

第 13 章:Claude Code 內建工具

13.1 工具選擇參考

任務	工具	範例
依名稱/模式尋找檔案	**Glob**	`**/*.test.tsx`、`src/components/**/*.ts`
在檔案內搜尋	**Grep**	函式名稱、錯誤訊息、import
完整讀取一個檔案	**Read**	載入檔案以供分析
寫入新檔案	**Write**	從零開始建立檔案
精準編輯既有檔案	**Edit**	透過唯一的文字比對替換特定片段
執行 shell 命令	**Bash**	git、npm、執行測試、建置

13.2 漸進式調查策略

不要一次讀取所有檔案。漸進式地建立理解:

```
1. Grep:尋找進入點(函式定義、export)
2. Read:讀取找到的檔案
3. Grep:尋找用法(import、呼叫)
4. Read:讀取使用端檔案
5. 重複,直到掌握完整全貌
```

13.3 後備方案:用 Read + Write 取代 Edit

當 Edit 因文字比對不唯一而失敗時:

1. Read — 載入完整的檔案內容
2. 以程式化方式修改內容
3. Write — 寫入更新後的版本

第二部分:考試領域筆記

領域 1:代理架構與編排(27%)

1.1 設計用於自主任務執行的代理迴圈

重點知識:

- 代理迴圈生命週期:發送 Claude 請求、檢查 `stop_reason`(`"tool_use"` 對比 `"end_turn"`)、執行工具、回傳結果以供下一次迭代使用
- 工具結果會附加到對話歷史中,讓模型能決定下一個動作
- 模型驅動的決策(由 Claude 選擇下一個工具)對比硬編碼的決策樹

重點技能:

- 流程控制:當 `stop_reason = "tool_use"` 時繼續迴圈,遇到 `"end_turn"` 時停止
- 在迭代之間將工具結果附加到上下文中
- 應避免的反模式:解析助理文字以判斷完成、將任意的迭代上限當作主要的停止機制

1.2 編排多代理系統(協調者—子代理)

重點知識:

- 中心輻射式架構:協調者掌管所有代理間的通訊、錯誤處理與路由
- 子代理在隔離的上下文中運作——它們不會自動繼承協調者的歷史
- 協調者職責:任務分解、委派、結果彙整、子代理的動態選擇
- 協調者過度狹隘分解的風險

重點技能:

- 在子代理之間分配研究範圍以盡量減少重複
- 實作迭代式精煉迴圈(協調者評估綜整結果並重新路由任務)
- 將所有通訊都透過協調者進行以利可觀測性

1.3 設定子代理呼叫、上下文傳遞與生成

重點知識:

- `Task` 工具用於生成子代理;協調者的 `allowedTools` 必須包含 `"Task"`
- 子代理的上下文必須明確包含在提示中;子代理不會繼承父層上下文
- `AgentDefinition` 設定:描述、系統提示、工具限制
- 透過 `fork_session` 進行工作階段管理以探索不同方案

重點技能:

- 在子代理提示中包含先前代理的完整輸出
- 在傳遞上下文時使用結構化格式,將資料與中繼資料分開
- 在單一協調者輪次中透過多個 `Task` 呼叫生成平行子代理
- 以目標與品質標準來撰寫協調者提示,而非逐步指令

1.4 以強制執行與交接模式實作多步驟工作流程

重點知識:

- **程式化強制執行**(hooks、前置條件)與**提示引導**在工作流程排序上的差異
- 當你需要確定性保證時(例如金融操作前的身分驗證),單靠提示是不夠的
- 升級期間的結構化交接協定(客戶 ID、原因、建議動作)

重點技能:

- 程式化前置條件,在先前步驟完成前阻擋下游呼叫(例如在 `get_customer` 回傳已驗證的 ID 之前阻擋 `process_refund`)
- 將多面向的客戶請求分解為個別項目
- 在升級至真人時產生結構化摘要

1.5 用於攔截工具呼叫與正規化資料的 Agent SDK Hooks

重點知識:

- Hook 模式(例如 `PostToolUse`),在模型取用工具結果之前加以攔截
- 攔截外送呼叫以強制執行合規規則的 Hooks(例如阻擋超過門檻的退款)
- Hooks 提供**確定性保證**,而提示指令僅提供**機率性合規**

重點技能:

- 用 `PostToolUse` hooks 正規化資料格式(Unix timestamps、ISO 8601、數值狀態碼)
- 攔截 hooks 以阻擋違反政策的動作,並重新導向至升級流程
- 當業務規則需要保證合規時,選擇 hooks 而非提示

1.6 複雜工作流程的任務分解策略

重點知識:

- **固定管線**(提示鏈接)對比根據中間結果進行的**動態自適應分解**
- 提示鏈接:循序步驟(分別分析每個檔案,再執行一次整合遍歷)
- 自適應調查計畫,根據所發現的內容產生子任務

重點技能:

- 對可預測的多面向審查使用提示鏈接;對開放式調查使用動態分解
- 將大型程式碼審查拆分為逐檔分析,加上獨立的跨檔整合遍歷
- 分解開放式任務:先盤點結構,再建立排定優先順序的計畫

1.7 工作階段狀態、恢復與分叉

重點知識:

- `--resume <session-name>` 用於繼續具名的工作階段
- `fork_session` 用於從共享上下文建立獨立的調查分支
- 在恢復工作階段時告知代理檔案變更的重要性
- 帶有結構化摘要的新工作階段,可能比帶著過時結果恢復更為可靠

重點技能:

- 使用 `--resume` 繼續具名的調查工作階段
- 使用 `fork_session` 平行比較不同做法
- 在恢復(上下文仍為最新)與啟動新工作階段(結果已過時)之間做出選擇

領域 2:工具設計與 MCP 整合(18%)

2.1 以清楚的描述設計工具介面

關鍵知識:

- 工具描述是 LLM 用來選擇工具的**主要機制**;描述過於精簡會導致選擇不可靠
- 納入輸入格式、範例查詢、邊界案例與適用範圍界線的重要性
- 模糊或重疊的描述會造成錯誤路由
- 系統提示的用字遣詞可能與工具產生非預期的關聯

關鍵技能:

- 撰寫能清楚區分各工具與相似替代工具的描述
- 重新命名工具以消除功能重疊(例如 `analyze_content` -> `extract_web_results`)

- 將通用型工具拆分成具有清楚輸入/輸出合約的專用工具

2.2 為 MCP 工具實作結構化錯誤回應

關鍵知識:

- MCP 工具回應中的 `isError` 旗標
- **暫時性錯誤**(逾時)、**驗證錯誤**(輸入錯誤)、**業務錯誤**(違反政策)與**存取/權限錯誤**之間的差異
- 通用錯誤(「Operation failed」)會妨礙做出正確的復原決策
- 可重試與不可重試錯誤之間的差異

關鍵技能:

- 回傳結構化中繼資料,例如 `errorCategory`(transient/validation/permission)、`isRetryable` 以及人類可讀的訊息
- 對違反業務規則的情況使用 `retryable: false`,並附上清楚的面向使用者的說明
- 對暫時性失敗在子代理內部進行本機復原;只將它們無法解決的錯誤向上傳遞
- 區分存取失敗(重試決策)與有效的空結果(無相符項目)

2.3 跨代理分配工具並設定 `tool_choice`

關鍵知識:

- 每個代理的工具過多(例如 18 個而非 4–5 個)會**降低**工具選擇的可靠性
- 擁有超出其專長範圍工具的代理往往會誤用它們
- 限定範圍的工具存取:只提供與角色相關的工具,加上一組有限的跨角色公用工具
- `tool_choice`:`"auto"`、`"any"` 以及強制工具選擇(`{"type": "tool", "name": "..."}`)

關鍵技能:

- 將每個子代理的工具集限制為與其角色相關的工具
- 以受約束的替代工具取代通用工具(例如 `fetch_url` -> `load_document`)
- 使用 `tool_choice: "any"` 以保證產生工具呼叫而非文字答案
- 強制使用特定工具以確保執行順序

2.4 將 MCP 伺服器整合至 Claude Code 與代理工作流程

關鍵知識:

- MCP 伺服器範圍:供團隊使用的專案層級(`.mcp.json`)vs 供實驗使用的使用者層級(`~/.claude.json`)
- 在 `.mcp.json` 中進行環境變數替換(例如 `${GITHUB_TOKEN}`)以管理機密
- 所有已連線 MCP 伺服器的工具會在連線時被探索並同時可用
- 將 MCP 資源作為「內容目錄」(任務摘要、資料庫結構描述)以減少探索性的工具呼叫

關鍵技能:

- 在專案 `.mcp.json` 中以環境變數為基礎的 token 設定共用的 MCP 伺服器
- 將個人/實驗性的伺服器保留在 `~/.claude.json` 中
- 對於標準整合,優先採用社群 MCP 伺服器而非自訂伺服器

2.5 選擇並運用內建工具(Read、Write、Edit、Bash、Grep、Glob)

關鍵知識:

- **Grep**:在檔案內容中搜尋(函式名稱、錯誤訊息、匯入)
- **Glob**:依名稱/副檔名模式尋找檔案
- **Read/Write**:整個檔案的操作;**Edit**:透過唯一文字比對進行精確變更
- 若 Edit 因比對不唯一而失敗,改用 Read + Write

關鍵技能:

- 使用 Grep 進行內容搜尋,使用 Glob 依模式探索檔案
- 漸進式建立理解:先 Grep 進入點,再 Read 追蹤流程
- 透過包裝模組追蹤函式的使用

領域 3:Claude Code 設定與工作流程(20%)

3.1 以階層、範圍與模組化組織設定 CLAUDE.md

關鍵知識:

- CLAUDE.md 階層:使用者(`~/.claude/CLAUDE.md`)、專案(`.claude/CLAUDE.md` 或根目錄的 `CLAUDE.md`)以及目錄層級(子目錄中的 CLAUDE.md)
- 使用者層級的設定只套用於單一使用者,且不會透過 VCS 共用
- 用於參照外部檔案的 `@path` 語法(例如 `@./standards/coding-style.md`),以模組化 CLAUDE.md
- 使用 `.claude/rules/` 目錄存放主題聚焦的規則檔案,而非單體式的 CLAUDE.md

關鍵技能:

- 診斷階層問題(新進團隊成員因為指令屬於使用者層級而非專案層級而錯過這些指令)
- 使用 `@path`(例如 `@./standards/testing.md`)以在每個套件的 CLAUDE.md 中選擇性納入標準
- 將龐大的 CLAUDE.md 拆分成多個 `.claude/rules/` 檔案(testing.md、api-conventions.md、deployment.md)

3.2 建立並設定自訂斜線命令與技能

關鍵知識:

- 位於 `.claude/commands/` 的**專案命令**(透過 VCS 共用)vs 位於 `~/.claude/commands/` 的**使用者命令**
- 位於 `.claude/skills/` 並具有 `SKILL.md` frontmatter 的技能:`context: fork`、`allowed-tools`、`argument-hint`
- `context: fork` 會在隔離的子代理上下文中執行技能,使其不會汙染主工作階段
- 個人化的技能變體可以不同名稱存放在 `~/.claude/skills/` 中

關鍵技能:

- 將專案斜線命令存放在 `.claude/commands/`,讓整個團隊都能取得
- 使用 `context: fork` 來隔離具有冗長輸出的技能
- 使用 `allowed-tools` 限制技能可使用的工具
- 使用 `argument-hint` 提示開發者輸入必要的參數

3.3 使用路徑專屬規則進行條件式慣例載入

關鍵知識:

- `.claude/rules/` 檔案可包含 YAML frontmatter `paths`,依據 glob 模式啟用規則
- 路徑範圍規則**僅**在編輯符合的檔案時載入,可節省上下文與 token
- 當慣例適用於許多目錄(例如測試)時,以 glob 為基礎的路徑規則可能優於目錄層級的 CLAUDE.md

關鍵技能:

- 建立帶有 `paths: ["terraform/**/*"]` 的 `.claude/rules/` 檔案,僅在處理符合的檔案時載入
- 使用 glob 模式(`**/*.test.tsx`)依檔案類型套用慣例,而不受位置影響
- 當慣例橫跨整個程式碼庫時,優先採用路徑專屬規則而非目錄層級的 CLAUDE.md

3.4 決定何時使用規劃模式而非直接執行

關鍵知識:

- **規劃模式**:適用於變更量大、有多種可行做法且涉及架構決策的複雜任務
- **直接執行**:適用於簡單、充分理解的變更(例如新增單一驗證)
- 規劃模式可在做出變更前安全地探索程式碼庫
- Explore 子代理可隔離冗長的探查輸出

關鍵技能:

- 對具有架構影響的任務使用規劃模式(微服務、觸及 45 個以上檔案的遷移)
- 對具有清楚堆疊追蹤且僅涉及單一檔案的修正使用直接執行
- 使用 Explore 子代理以防止多階段任務耗盡上下文視窗
- 結合兩種做法:以規劃進行探查,接著以執行進行實作

3.5 透過迭代精修達成漸進式改善

關鍵知識:

- 具體的輸入/輸出範例是傳達期望最有效的方式
- **測試驅動迭代**:先撰寫測試,再依失敗結果進行迭代
- 「面談」模式:Claude 提出問題,以浮現非顯而易見的設計考量
- 何時在單一訊息中提供所有問題(彼此相依)vs 依序提供(彼此獨立)

關鍵技能:

- 提供 2–3 個具體的輸入/輸出範例,以釐清轉換需求
- 在實作前建立測試集,涵蓋預期行為、邊界案例與效能需求
- 使用面談模式以浮現設計面向(快取失效、失敗模式)
- 針對邊界案例提供具體的測試案例,含範例輸入與預期輸出

3.6 將 Claude Code 整合進 CI/CD 管線

關鍵知識:

- 在自動化管線中用於非互動模式的 `-p`(或 `--print`)旗標
- 在 CI 中用於結構化輸出的 `--output-format json` 與 `--json-schema`
- CLAUDE.md 為 CI 觸發的 Claude Code 提供專案上下文(測試標準、審查準則)
- **工作階段上下文隔離**:生成程式碼的同一個工作階段,在審查該程式碼時不如獨立執行個體有效

關鍵技能:

- 在 CI 中以 `-p` 執行 Claude Code,以避免因互動式輸入而卡住
- 使用 `--output-format json` + `--json-schema` 取得結構化結果(例如行內 PR 留言)
- 在有新提交後重新執行時納入先前的審查結果(僅回報新的/未修正的問題)
- 在 CLAUDE.md 中記錄測試標準與可用的 fixtures,以提升測試生成品質
- 生成新測試時將既有測試檔案納入上下文,以避免重複並維持風格一致

領域 4:提示工程與結構化輸出(20%)

4.1 以明確準則設計提示以提升準確度

關鍵知識:

- 明確準則比含糊指令更有效(例如「僅在註解與程式碼相矛盾時才標記」vs「檢查註解準確度」)
- 像「更保守一點」這類泛用指引,效果不如具體的分類準則
- 誤報對開發者信任的影響:某些類別的高誤報率會破壞對準確類別的信任

關鍵技能:

- 定義審查準則:應回報什麼(bug、安全性)vs 應忽略什麼(輕微風格問題)
- 暫時停用誤報率高的類別
- 為每個等級定義明確的嚴重度準則,並附上程式碼範例

4.2 使用少樣本提示以提升輸出一致性

關鍵知識:

- 少樣本範例是產生格式一致、可付諸行動的輸出最有效的方法
- 少樣本可示範如何處理模糊案例(工具選擇、測試涵蓋率的缺口)
- 少樣本協助模型推廣到新模式,而不僅是重複預設行為
- 少樣本可降低擷取任務中的幻覺

關鍵技能:

- 針對模糊情境提供 2–4 個有針對性的範例並附上理由
- 納入示範輸出格式的少樣本範例(位置、問題、嚴重度、建議修正)
- 提供範例以區分可接受的程式碼模式與真正的問題
- 提供從不同結構文件中正確擷取的範例

4.3 以 `tool_use` 與 JSON Schemas 強制結構化輸出

關鍵知識:

- 搭配 JSON Schemas 的 `tool_use` 是保證輸出符合結構描述並消除 JSON 語法錯誤最可靠的方式
- 在 `tool_choice: "auto"` 下模型可回傳文字;在 `"any"` 下則必須呼叫工具;強制選擇則會選定特定工具
- 嚴格的 JSON Schemas 可消除語法錯誤,但無法防止語意錯誤(總計加總不對;值放在錯誤欄位)
- 結構描述設計:必填 vs 選填欄位;列舉值加上「other」並搭配細節字串以利擴充

關鍵技能:

- 以 JSON Schemas 定義擷取工具,並從 `tool_use` 結果解析資料
- 當存在多個結構描述時,使用 `tool_choice: "any"` 以保證結構化輸出
- 強制特定的工具呼叫:`tool_choice: {"type": "tool", "name": "extract_metadata"}`
- 當來源可能不含某項資訊時,將欄位設為選填/可為 null,以避免捏造值
- 使用像 `"unclear"` 與 `"other"` 這類列舉值並搭配細節欄位,以利可擴充的分類

4.4 為擷取品質實作驗證、重試與回饋迴圈

關鍵知識:

- 帶錯誤回饋的重試:在重試提示中納入具體的驗證錯誤,以引導修正
- 當資訊根本不存在於來源中時,重試無效

- 回饋迴圈設計:追蹤觸發某個發現的模式(`detected_pattern`)
- 語意錯誤(總計無法對帳)相對於語法錯誤(由 `tool_use` 處理)

關鍵技能:

- 撰寫後續提示,附上原始文件、一份不正確的擷取結果,以及具體的驗證錯誤
- 辨識何時重試將無效(所需資訊只存在於外部文件中)
- 在發現中納入 `detected_pattern` 欄位,以分析誤報
- 透過同時擷取 `calculated_total` 與 `stated_total` 來偵測差異,設計自我修正

4.5 設計高效的批次處理策略

關鍵知識:

- Message Batches API:節省 50%、最長 24 小時的處理視窗、不保證延遲 SLA
- 批次處理適用於非阻斷式任務(隔夜報表、稽核),不適用於阻斷式任務(合併前檢查)
- 批次 API 不支援在單一請求內進行多輪工具呼叫
- `custom_id` 欄位用於在批次內關聯請求與回應

關鍵技能:

- 阻斷式檢查使用同步 API;隔夜/每週工作負載使用批次 API
- 依 SLA 需求規劃批次提交頻率(例如以 4 小時視窗搭配 24 小時處理,達成 30 小時保證)
- 處理失敗時只重新提交失敗的文件(以 `custom_id` 辨識)
- 在大規模處理之前,先用樣本反覆調整提示

4.6 設計多執行個體與多遍審查架構

關鍵知識:

- 自我審查的侷限:模型保留了自身的推理上下文,較不可能挑戰自己的決定
- 獨立的審查執行個體(不帶生成上下文)更擅長找出細微問題
- 多遍審查:每檔案的局部分析,加上跨檔案的整合遍,以避免注意力被稀釋

關鍵技能:

- 使用第二個獨立的 Claude 執行個體,在不帶生成上下文的情況下審查變更
- 將多檔案審查拆成每檔案遍,加上整合遍,以進行跨檔案的資料流分析
- 使用帶自我評定信心的驗證遍,以校準的方式路由審查

領域 5:上下文管理與可靠性(15%)

5.1 管理對話上下文以保留關鍵資訊

關鍵知識:

- 漸進式摘要的風險:數值、百分比與日期會被濃縮成模糊的摘要
- 迷失在中間效應:模型能可靠處理長輸入的開頭與結尾,但可能漏掉中間的發現
- 工具輸出在上下文中累積的比例可能與其相關性不成正比(需要 5 個欄位時卻有 40 多個)
- 在後續 API 請求中傳送完整對話歷史的重要性

關鍵技能:

- 將交易事實擷取到摘要歷史之外的持久「案件事實」區塊中
- 將冗長的工具輸出修剪到只剩相關欄位
- 將關鍵發現放在彙整資料的開頭,並加上明確的章節標題
- 要求子代理在結構化輸出中納入中繼資料(日期、來源)

5.2 設計有效的升級模式並解決不明確性

關鍵知識:

- 合適的升級觸發條件:明確要求真人、政策缺口/例外、無法取得進展
- 立即升級(明確要求)相對於嘗試解決(在代理職權範圍內)
- 情緒分析與模型信心自我評定,都不是案件複雜度的可靠替代指標
- 比對到多筆客戶時應要求額外的識別資訊,而非以啟發法猜測

關鍵技能:

- 在系統提示中以少樣本範例列出明確的升級準則
- 立即執行明確要求真人的請求,不做額外調查
- 當政策對特定請求不明確或未著墨時,進行升級
- 當工具結果包含多筆比對時,要求額外的識別資訊

5.3 在多代理系統中實作錯誤傳播策略

關鍵知識:

- 結構化的錯誤上下文(失敗類型、查詢、部分結果、替代方案)能讓協調者更聰明地復原
- 區分存取失敗(逾時需要重試決策)與有效的空結果(無相符項目)
- 籠統的錯誤狀態(「搜尋無法使用」)會對協調者隱藏有價值的上下文
- 在單一失敗時靜默抑制或中止整個工作流程,兩者都是反模式

關鍵技能:

- 回傳結構化的錯誤上下文:失敗類型、嘗試了什麼、部分結果、可能的替代方案
- 區分存取失敗與有效的空結果
- 在子代理內對暫時性失敗進行局部復原;只將不可復原的錯誤連同部分結果向上傳播
- 在綜整時標註涵蓋範圍:哪些有充分佐證、哪些仍有缺口

5.4 調查大型程式碼庫時的高效上下文管理

關鍵知識:

- 長工作階段中的上下文劣化:模型開始產生不穩定的答案,並提到「典型模式」而非具體類別
- 暫存檔(scratchpad)可跨上下文邊界保留關鍵發現
- 委派給子代理可隔離冗長的探索輸出
- 結構化的狀態持久化能在當機後復原

關鍵技能:

- 為特定問題生成子代理,同時在主代理中保留高層級的協調
- 使用暫存檔(scratchpad)儲存關鍵發現並於稍後參照
- 在生成下一階段子代理之前,先摘要關鍵發現
- 在長時間調查期間使用 `/compact` 以降低上下文用量

5.5 設計具備人工監督與信心校準的工作流程

關鍵知識:

- 彙總指標(例如 97% 的整體準確率)可能掩蓋在特定文件類型或欄位上的不佳表現
- 分層隨機抽樣可衡量高信心擷取結果的錯誤率
- 使用已標註的驗證集進行欄位層級的信心校準
- 在自動化之前,依文件類型與欄位區段驗證準確率

關鍵技能:

- 實作分層隨機抽樣以偵測新的錯誤模式
- 依文件類型與欄位分析準確率,以驗證穩定的表現
- 輸出欄位層級的信心分數,並使用已標註的資料校準審查門檻
- 將低信心或來源不明確的擷取結果路由至人工審查

5.6 在多來源綜整中保留出處來源並處理不確定性

關鍵知識:

- 在摘要過程中若未保留「主張 → 來源」對應,出處標註便會遺失
- 在彙整過程中必須保留結構化的對應關係

- 處理衝突的統計數據時,應以出處標註來註記衝突,而非任意選擇某一個值
- 納入發布/收集日期,以避免將時間差異誤讀為矛盾

關鍵技能:

- 要求子代理輸出「主張 → 來源」對應(URL、文件名稱、引文)
- 將報告結構化,以區分穩定的發現與有爭議的發現
- 保留衝突的值並加上註記,再傳遞給協調者進行調和
- 納入發布日期以利正確的時間解讀
- 依類型呈現內容:財務資料以表格呈現、新聞以散文呈現、技術發現以結構化清單呈現

考題範例與解析

問題 1(情境:客戶支援代理)

情境: 資料顯示,在 12% 的案例中,代理跳過 `get_customer`,僅以客戶的姓名呼叫 `lookup_order`,進而導致錯誤的退款。

哪一項變更最為有效?

- A) 加入程式化的前置條件,在從 `get_customer` 取得 ID 之前封鎖 `lookup_order` 與 `process_refund` **[CORRECT]**
- B) 改善系統提示
- C) 加入少樣本範例
- D) 實作路由分類器

為何選 A: 當關鍵業務邏輯需要特定的工具順序時,軟體能提供以提示為基礎的做法(B、C)所無法提供的**確定性保證**。D 處理的是可用性,而非工具排序。

問題 2(情境:客戶支援代理)

情境: 對於與訂單相關的問題,代理經常呼叫 `get_customer` 而非 `lookup_order`。工具描述極為精簡且彼此相似。

第一步該做什麼?

- A) 少樣本範例
- B) 為每個工具的描述補充輸入格式、範例與邊界 **[CORRECT]**
- C) 加入路由層
- D) 合併這些工具

為何選 B: 工具描述是模型主要的選擇機制。這是投入最少、影響最大的修正。A 增加了 tokens,卻未處理根本原因。C 是過度設計。D 所需投入超過其合理性。

問題 3(情境:客戶支援代理)

情境: 代理僅解決了 55% 的問題,而目標為 80%。它將簡單的案例升級,卻試圖自主處理複雜的政策例外。

你該如何改善校準?

- A) 加入明確的升級準則並搭配少樣本範例 **[CORRECT]**
- B) 自評信心(1–10)並自動升級
- C) 以歷史資料訓練的獨立分類器
- D) 情感分析

為何選 A: 它直接處理了根本原因——不明確的決策邊界。B 不可靠(模型可能在充滿信心的同時卻是錯的)。C 是過度設計。D 解決的是另一個問題(情緒 != 複雜度)。

問題 4(情境:使使用 Claude Code 生成程式碼)

情境: 你需要一個用於標準程式碼審查的自訂 `/review` 命令,並讓整個團隊在複製儲存庫時都能使用。

你應該在哪裡建立這個命令檔案?

- A) 專案儲存庫中的 `.claude/commands/` **[CORRECT]**
- B) `~/.claude/commands/`
- C) 根目錄的 `CLAUDE.md`
- D) `.claude/config.json`

為何選 A: 儲存於 `.claude/commands/` 的專案命令會納入版本控制,並自動供所有人使用。B 用於個人命令。C 用於指令,而非命令定義。D 並不存在。

問題 5(情境:使使用 Claude Code 生成程式碼)

情境: 你需要將一個單體應用程式重構為微服務(數十個檔案、服務邊界的決策)。

你應該採用什麼做法?

- A) 規劃模式:探索程式碼庫、理解相依性、設計做法 **[CORRECT]**
- B) 漸進式的直接執行
- C) 搭配詳盡前置指令的直接執行
- D) 直接執行,等到變困難時再切換至規劃

為何選 A: 規劃模式專為大型變更、多種可能做法與架構決策而設計。B 有昂貴重工的風險。C 假設你已經知道結構。D 是被動反應式的。

問題 6(情境:使使用 Claude Code 生成程式碼)

情境: 某個程式碼庫在不同範圍(React、API、資料庫)有不同的慣例。測試與程式碼放在同一處。你希望這些慣例能自動套用。

你應該採用哪種做法?

- A) 使用帶有 YAML frontmatter 與 glob 模式的 `.claude/rules/` 檔案 **[CORRECT]**
- B) 把所有內容都放進根目錄的 CLAUDE.md
- C) 放在 `.claude/skills/` 的 Skills
- D) 在每個目錄都放一份 CLAUDE.md

為何選 A: 帶有 glob 模式(例如 `**/*.test.tsx`)的 `.claude/rules/` 能根據檔案路徑自動套用慣例——非常適合分散在整個程式碼庫中的測試。B 仰賴模型推論。C 是手動/隨需的。D 在相關檔案散落於多個目錄時運作不佳。

問題 7(情境:多代理研究系統)

情境: 該系統研究「AI 對創意產業的影響」,但報告只涵蓋視覺藝術。協調者將主題分解為:「數位藝術中的 AI」、「平面設計中的 AI」、「攝影中的 AI」。

原因是什麼?

- A) 綜整代理沒有偵測到缺口
- B) 協調者把任務分解得太狹隘 **[CORRECT]**
- C) 網路搜尋代理搜尋得不夠徹底
- D) 文件分析代理過濾掉了非視覺類的來源

為何選 B: 紀錄顯示協調者把「創意產業」只分解成視覺類的子主題,完全漏掉了音樂、文學與電影。子代理執行正確——問題在於它們被指派的任務內容。

問題 8(情境:多代理研究系統)

情境: 某個網路搜尋子代理在研究一個複雜主題時逾時。你需要設計錯誤資訊如何回傳給協調者。

哪種錯誤傳播做法最能促成智慧型復原?

- A) 將結構化的錯誤上下文回傳給協調者:失敗類型、查詢、部分結果與替代方案 **[CORRECT]**
- B) 在子代理內實作帶有指數退避的自動重試,然後回傳一個籠統的「search unavailable」狀態
- C) 在子代理內捕捉逾時,並回傳一個標記為成功的空結果集
- D) 將逾時例外向上傳播至最上層的處理器,並終止整個工作流程

為何選 A: 結構化的錯誤上下文提供協調者所需的資訊,以決定是否要用修改後的查詢重試、嘗試替代做法,或以部分結果繼續。B 把上下文藏在籠統的狀態後面。C 把失敗偽裝成成功。D 不必要地中止了整個工作流程。

問題 9(情境:多代理研究系統)

情境: 綜整代理在合併結果時經常需要驗證特定主張。目前,當需要驗證時,綜整代理會把控制權交回協調者,由協調者呼叫網路搜尋代理,然後用新結果重新執行綜整。這每個任務會增加 2–3 次額外的往返,並使延遲增加 40%。你的評估顯示,其中 85% 的檢查是簡單的事實查核(日期、姓名、統計數字),而 15% 需要更深入的調查。

你要如何在維持可靠性的同時降低額外開銷?

- A) 給綜整代理一個受限的 `verify_fact` 工具來處理簡單的查核,並繼續讓複雜的驗證透過協調者路由 **[CORRECT]**
- B) 把所有驗證需求累積成一個批次,在最後回傳給協調者
- C) 給綜整代理對所有網路搜尋工具的完整存取權
- D) 主動快取每個來源周遭的額外上下文

為何選 A: 這套用了最小權限原則:綜整代理恰好取得它在 85% 常見情況(簡單事實查核)所需的東西,同時保留由協調者中介的路徑來處理複雜的調查。B 引入了阻塞式相依性(後續的綜整步驟可能仰賴先前已驗證的事實)。C 破壞了職責分離。D 仰賴推測性快取,而這無法可靠地預測需求。

問題 10(情境:用於持續整合的 Claude Code)

情境: 某條管線執行 `claude "Analyze this pull request for security issues"`,卻卡住等待互動式輸入。

正確的做法是什麼?

- A) 使用 `-p` 旗標:`claude -p "Analyze this pull request for security issues"` **[CORRECT]**
- B) 設定 `CLAUDE_HEADLESS=true`
- C) 將 stdin 從 `/dev/null` 重新導向
- D) 使用 `--batch`

為何選 A: `-p`(或 `--print`)是有文件記載、用於以非互動模式執行 Claude Code 的方式。它會處理提示、印出到 stdout,然後結束。其他選項不是不存在的功能,就是 Unix 的權宜做法。

問題 11(情境:用於持續整合的 Claude Code)

情境: 團隊想要降低自動化分析的 API 成本。Claude 目前即時服務兩種工作流程:(1) 一個阻塞式的合併前檢查,必須在開發者能夠合併 PR 之前完成;以及 (2) 一份在夜間產生、供早上審閱的技術債報告。一位主管提議把兩者都移到 Message Batches API 以節省 50%。

你應該如何評估這項提案?

- A) 只對技術債報告使用批次處理;合併前檢查維持即時呼叫 **[CORRECT]**
- B) 把兩個工作流程都移到批次處理,並輪詢完成狀態
- C) 兩者都維持即時呼叫,以避免批次結果中的排序問題
- D) 把兩者都移到批次處理,並在某個批次耗時過久時退回即時呼叫

為何選 A: Message Batches API 可節省 50%,但處理時間最長可達 24 小時,且沒有保證的延遲 SLA。這使它不適合開發者正在等待的阻塞式合併前檢查,但非常適合像技術債報告這類的夜間批次工作負載。

問題 12(情境:多檔案程式碼審查)

情境: 某個 pull request 變更了一個庫存追蹤模組中的 14 個檔案。對所有檔案進行單遍審查會產生不一致的結果:某些檔案有詳細的評論,其他檔案卻很表面,漏掉明顯的 bug,還有互相矛盾的回饋(某個模式在一個檔案被標記為有問題,但在另一個檔案中相同的程式碼卻被核可)。

你應該如何重新建構這次審查?

- A) 拆成聚焦的多遍:逐一分析每個檔案的局部問題,然後執行一次獨立的整合遍來檢查跨檔案的資料流 **[CORRECT]**
- B) 要求開發者把大型 PR 拆成每次提交 3–4 個檔案
- C) 改用具有更大上下文視窗的更高階模型,在一遍之內審查全部 14 個檔案
- D) 執行三次獨立的完整 PR 審查遍次,只回報在至少兩次執行中都發現的問題

為何選 A: 聚焦的多遍直接針對根本原因——一次處理大量檔案時的注意力稀釋。逐檔分析確保一致的深度,而獨立的整合遍能捕捉跨檔案的問題。B 把負擔轉嫁給開發者,卻沒有改善系統。C 是一種誤解:更大的上下文無法修正注意力的品質。D 透過要求在不一致的偵測之間取得共識,壓制了真正的 bug。

練習測驗

> 涵蓋 4 個情境、共 60 道考題。格式與難度皆比照正式考試。
>
> 此外,你也可以在類似考試的 HTML 檔案中練習這些考題:Practical Test (EN)

情境:多代理研究系統

問題 1(情境:多代理研究系統)

情境: 某文件分析代理發現兩個可信來源對於一項關鍵指標含有直接相互矛盾的統計數字:一份政府報告指出成長率為 40%,而一份產業分析則指出為 12%。兩個來源看起來都很可信,且此落差可能對研究結論造成實質影響。該文件分析代理應如何最有效地處理此情況?

哪種做法最有效?

- A) 套用可信度的經驗法則挑出最可能正確的數字,以該值完成分析,並加註腳說明此落差。
- B) 在分析輸出中同時納入兩個數字,但不標記它們相互衝突,讓綜整代理根據更廣的上下文決定要採用哪一個。
- C) 停止分析並立即升級至協調者,請它在繼續之前決定哪個來源更具權威性。

- D) 以兩個數字一併完成分析,明確標註此衝突並附上來源出處標註,在傳遞給綜整之前,讓協調者決定如何調和這些資料。 **[CORRECT]**

為何選 D: 此做法維持了職責分離:分析代理在不阻塞的情況下完成其核心工作,以清楚的出處標註保留兩個相互衝突的值,並正確地將調和工作交給擁有更廣上下文的協調者。

問題 2(情境:多代理研究系統)

情境: 網頁搜尋代理與文件分析代理已完成各自的任務,並將結果回傳給協調者。建立整合研究報告的下一步是什麼?

下一步哪個最適當?

- A) 每個代理將其結果直接傳送給報告撰寫代理,略過協調者。
- B) 文件分析代理向網頁搜尋代理索取搜尋結果,並在內部進行合併。
- C) 協調者將兩組結果傳遞給綜整代理,進行統一整合。 **[CORRECT]**
- D) 協調者將兩個代理的原始輸出串接起來,並將其作為最終結果回傳。

為何選 C: 在協調者—子代理架構中,協調者將兩組結果轉交給綜整代理進行集中整合,既保有控制權,也確保高品質的合併。

問題 3(情境:多代理研究系統)

情境: 某文件分析子代理在處理 PDF 檔案時經常失敗:有些檔案含有損毀的區段,會觸發剖析例外;有些則受密碼保護;有時剖析函式庫在處理大型檔案時會卡住。目前,任何例外都會立即終止子代理,並向協調者回傳一個錯誤,協調者必須決定要重試、略過,還是讓整個任務失敗。這導致協調者過度涉入例行的錯誤處理。哪項架構改進最有效?

哪項改進最有效?

- A) 建立一個專責的錯誤處理代理,透過共用佇列監控所有失敗並決定復原動作,直接向子代理發送重啟命令。
- B) 設定子代理一律回傳帶有成功狀態的部分結果,並將錯誤細節嵌入中繼資料;協調者將所有回應都視為成功。
- C) 讓協調者在將文件傳送給子代理之前先驗證所有文件,拒絕那些可能導致失敗的文件。
- D) 在子代理內針對暫時性失敗實作本機復原,僅將其無法解決的錯誤升級至協調者,並附上嘗試過的步驟與部分結果。 **[CORRECT]**

為何選 D: 在有能力解決錯誤的最低層級處理錯誤。本機復原可減輕協調者的工作負擔,同時仍能將真正無法復原的問題連同完整上下文與部分進度一併升級。

問題 4(情境:多代理研究系統)

情境: 在以「AI 對創意產業的影響」執行系統後,你觀察到每個子代理都成功完成:網頁搜尋代理找到相關文章,文件分析代理正確地將它們摘要,綜整代理也產出了連貫的文字。然而,最終報告只涵蓋視覺藝術,完全遺漏了音樂、文學與電影。在協調者的記錄檔中,你看到它將主題分解為三個子任務:「AI 於數位藝術」、「AI 於平面設計」與「AI 於攝影」。最可能的根本原因是什麼?

最可能的根本原因是什麼?

- A) 綜整代理缺少偵測涵蓋範圍缺口的指令。
- B) 文件分析代理因相關性標準過於嚴格,而濾除了非視覺類的來源。
- C) 協調者的任務分解過於狹隘,指派給子代理的工作未涵蓋所有相關領域。 **[CORRECT]**
- D) 網頁搜尋代理的查詢不足,應加以擴大以涵蓋更多領域。

為何選 C: 協調者只將一個廣泛的主題分解為視覺藝術相關的子任務,完全遺漏了音樂、文學與電影。由於子代理都正確執行了各自的指派,狹隘的分解便是顯而易見的根本原因。

問題 5(情境:多代理研究系統)

情境: 網頁搜尋子代理只回傳了 5 個請求來源類別中的 3 個(競爭對手網站與產業報告成功,但新聞檔案庫與社群動態逾時)。文件分析子代理成功處理了所有提供的文件。綜整子代理必須從品質不一的上游輸入產出摘要。哪種錯誤傳播策略最有效?

哪種錯誤傳播策略最有效?

- A) 僅使用成功的來源繼續綜整,並產出一份未提及哪些資料無法取得的輸出。
- B) 綜整子代理向協調者回傳一個錯誤,因資料不完整而觸發完整重試或任務失敗。
- C) 綜整子代理請協調者在開始綜整之前,以更長的逾時時間重試逾時的來源。
- D) 在綜整輸出中加入涵蓋範圍標註,指出哪些結論有充分佐證,以及哪些地方因來源無法取得而存在缺口。 **[CORRECT]**

為何選 D: 涵蓋範圍標註以透明的方式實作了優雅降級,既保留了已完成工作的價值,又能傳播不確定性,以利針對信心程度做出知情的決定。

問題 6(情境:多代理研究系統)

情境: 文件分析子代理遇到一個它無法剖析的損毀 PDF 檔案。在設計系統的錯誤處理時,處理此失敗最有效的方式是什麼?

哪種做法最有效?

- A) 向協調者代理回傳一個帶有上下文的錯誤,讓它決定如何繼續進行。 **[CORRECT]**
- B) 默默略過該損毀文件並繼續處理其餘檔案,以避免中斷工作流程。
- C) 在回報失敗之前,以指數退避自動重試剖析該文件三次。
- D) 拋出一個終止整個研究工作流程的例外。

為何選 A: 向協調者回傳一個帶有上下文的錯誤是最有效的做法,因為它讓協調者能做出知情的決定——略過該檔案、嘗試替代的剖析方法,或通知使用者——同時保持對該失敗的可見性。

問題 7(情境:多代理研究系統)

情境: 正式環境日誌顯示一個持續出現的模式:像「分析上傳的季度報告」這類請求,有 45% 的機率被路由到網頁搜尋代理,而不是文件分析代理。檢視工具定義後,你發現網頁搜尋代理有一個工具 `analyze_content`,描述為「analyzes content and extracts key information」,而文件分析代理有一個工具 `analyze_document`,描述為「analyzes documents and extracts key information」。你該如何修正這個路由錯誤的問題?

你該如何修正這個路由錯誤的問題?

- A) 加入一個前置路由分類器,在協調者決定委派之前,先偵測使用者指的是上傳的檔案還是網頁內容。
- B) 將網頁搜尋工具重新命名為 `extract_web_results`,並將其描述更新為「processes and returns information retrieved from web search and URLs」。 **[CORRECT]**
- C) 在協調者提示中加入少樣本範例,展示正確的路由:「使用者上傳季度報告 → 文件分析代理」以及「使用者詢問某個網頁 → 網頁搜尋代理」。
- D) 用使用範例擴充文件分析工具的描述,例如「用於上傳的 PDF、Word 文件和試算表」,而網頁搜尋工具則維持不變。

為何選 B: 將網頁搜尋工具重新命名為 `extract_web_results`,並更新其描述以明確指涉網頁搜尋與 URL,這直接消除了兩個工具名稱與描述之間的語意重疊,從而根除問題的根本原因。這使每個工具的用途都明確無歧義,讓協調者能可靠地區分文件分析與網頁搜尋。

問題 8(情境:多代理研究系統)

情境: 一位同事提議讓文件分析代理直接將其結果送給綜整代理,繞過協調者。將協調者保留為所有子代理之間通訊的中央樞紐,主要優勢是什麼?

將協調者保留為中央樞紐的主要優勢是什麼?

- A) 協調者可以觀察所有互動、統一處理錯誤,並決定每個子代理應該接收哪些資訊。 **[CORRECT]**
- B) 協調者將對子代理的多個請求批次化,減少 API 呼叫總數與整體延遲。
- C) 透過協調者路由能啟用直接代理間呼叫無法支援的自動重試邏輯。
- D) 子代理使用隔離的記憶體,直接通訊會需要複雜的序列化,而只有協調者能執行序列化。

為何選 A: 協調者模式提供對所有互動的集中可見性、整個系統一致的錯誤處理,以及對每個子代理接收哪些資訊的細緻控制——這些正是星形通訊拓撲的主要優勢。

問題 9(情境:多代理研究系統)

情境: 網頁搜尋子代理在研究一個複雜主題時逾時。你需要設計如何將這次失敗的相關資訊回傳給協調者。哪種錯誤傳播方式最能促成智慧型的復原?

哪種錯誤傳播方式最能促成智慧型的復原?

- A) 將結構化的錯誤上下文回傳給協調者,包含失敗類型、執行的查詢、任何部分結果,以及可能的替代方法。 **[CORRECT]**
- B) 在子代理內部攔截逾時,並回傳一個標記為成功的空結果集。
- C) 在子代理內部實作自動指數退避重試,只有在重試耗盡後才回傳一個一般性的「search unavailable」狀態。
- D) 將逾時例外直接傳播到最上層的處理常式,終止整個研究工作流程。

為何選 A: 回傳結構化的錯誤上下文——包含失敗類型、執行的查詢、部分結果以及替代方法——給予協調者做出智慧型復原決策所需的一切(例如,以修改後的查詢重試,或以部分結果繼續)。它為協調層級的明智決策保留了最大量的上下文。

問題 10(情境:多代理研究系統)

情境: 在你的系統設計中,你讓文件分析代理可以存取一個通用工具 `fetch_url`,以便它能透過 URL 下載文件。正式環境日誌顯示,這個代理現在經常下載搜尋引擎結果頁面來執行臨時的網頁搜尋——這種行為本應路由到網頁搜尋代理——導致結果不一致。哪種修正最有效?

哪種修正最有效?

- A) 將 `fetch_url` 替換為一個 `load_document` 工具,該工具會驗證 URL 是否指向文件格式。 **[CORRECT]**
- B) 從文件分析代理移除 `fetch_url`,並將所有 URL 抓取透過協調者路由到網頁搜尋代理。
- C) 實作過濾機制,封鎖對已知搜尋引擎網域的 `fetch_url` 呼叫,同時允許其他 URL。
- D) 在文件分析代理的提示中加入指令,說明 `fetch_url` 只應用於下載文件 URL,而非搜尋。

為何選 A: 將通用工具替換為一個會驗證 URL 是否符合文件格式的文件專用工具,透過在介面層級約束能力來修正根本原因。這遵循最小權限原則,使不想要的搜尋行為變成不可能,而非僅僅是不鼓勵。

問題 11(情境:多代理研究系統)

情境: 在研究一個廣泛主題時,你觀察到網頁搜尋代理與文件分析代理調查了相同的子主題,導致它們的輸出有大量重複。token 用量幾乎翻倍,但研究的廣度或深度卻沒有相應地增加。處理這個問題最有效的方式是什麼?

處理這個問題最有效的方式是什麼?

- A) 讓兩個代理平行完成,然後由協調者在將結果傳給綜整代理之前,先對重疊的結果去重。
- B) 協調者在委派之前明確劃分研究空間,為每個代理分配不同的子主題或來源類型。 **[CORRECT]**
- C) 實作一個共享狀態機制,讓代理記錄其當前的關注領域,使其他代理能在執行期間動態地避免重複。
- D) 改用循序執行,讓文件分析只在網頁搜尋完成後才執行,並以網頁搜尋結果作為上下文以避免重複。

為何選 B: 讓協調者在委派之前明確劃分研究空間最為有效,因為它在任何工作開始之前就處理了根本原因——任務邊界不明確。它在防止重複工作與浪費 token 的同時,保留了平行性。

問題 12(情境:多代理研究系統)

情境: 在研究期間,網頁搜尋子代理查詢了三個來源類別,結果各不相同:學術資料庫回傳 15 篇相關論文,產業報告回傳「0 results」,而專利資料庫回傳「Connection timeout」。在設計向協調者的錯誤傳播時,哪種方式能促成最佳的復原決策?

哪種方式能促成最佳的復原決策?

- A) 將結果彙整成單一的成功百分比指標(例如「67% source coverage」),並提供可隨需取用的詳細日誌。
- B) 將「timeout」和「0 results」都回報為需要協調者介入的失敗。
- C) 在內部重試暫時性失敗,只回報持續性的錯誤。
- D) 區分需要重試決策的存取失敗(timeout)與代表查詢成功的有效空結果(「0 results」)。 **[CORRECT]**

為何選 D: 逾時(存取失敗)與「0 results」(有效的空結果)在語意上是不同的結果,需要不同的回應。區分它們能讓協調者重試專利資料庫,同時接受產業報告的「0 results」為有效且有資訊價值的發現。

問題 13(情境:多代理研究系統)

情境: 正式環境監控顯示綜整品質不一致。當彙整後的結果約為 75K tokens 時,綜整代理能可靠地引用前 15K tokens(網頁搜尋的標題/摘錄)和最後 10K tokens(文件分析的結論)中的資訊,但經常遺漏中間 50K tokens 裡的關鍵發現——即使這些發現直接回答了研究問題。你該如何重新組織彙整後的輸入?

你該如何重新組織彙整後的輸入?

- A) 在彙整之前,將所有子代理的輸出摘要到 20K tokens 以下,以使內容保持在模型可靠的處理範圍內。
- B) 將子代理結果以增量方式串流給綜整代理,先將網頁搜尋結果處理到完成,再加入文件分析結果。
- C) 在彙整後輸入的開頭放置一份關鍵發現摘要,並以明確的章節標題組織詳細結果以利瀏覽。 **[CORRECT]**
- D) 實作輪替機制,在不同研究任務間交替讓哪一個子代理的結果出現在最前面,以確保兩個來源長期下來都能獲得同等的頂端位置。

為何選 C: 在開頭放置關鍵發現摘要能善用初始效應,使關鍵資訊位於處理最可靠的位置。在通篇加入明確的章節標題有助於模型瀏覽並關注輸入中段的內容,直接緩解「迷失在中間」的現象。

問題 14(情境:多代理研究系統)

情境: 在測試中,web-search 代理的輸出(85K tokens,包含頁面內容)與文件分析代理的輸出(70K tokens,包含思考鏈)合計共 155K tokens,但 綜整代理在輸入低於 50K tokens 時表現最佳。哪個解決方案最有效?

哪個解決方案最有效?

- A) 修改上游代理,使其回傳結構化資料(關鍵事實、引述、相關性分數),而非冗長的內容與推理過程。 **[CORRECT]**
- B) 新增一個中介摘要代理,在將結果傳給 綜整 之前先加以精簡。
- C) 讓 綜整代理以連續批次的方式處理結果,並在各次呼叫之間維持狀態。
- D) 將結果儲存在向量資料庫中,並給 綜整代理搜尋工具,使其能在工作過程中查詢。

為何選 A: 修改上游代理回傳結構化資料,能在源頭減少 token 量並保留必要資訊,從而解決根本原因。它避免傳遞龐大的頁面內容與推理軌跡,這些內容會膨脹 tokens 卻無助於 綜整 步驟。

問題 15(情境:多代理研究系統)

情境: 在測試中,你觀察到 綜整代理在合併結果時經常需要驗證特定主張。目前當需要驗證時,綜整代理會將控制權交還給協調者,協調者再呼叫 web-search 代理,然後帶著結果重新呼叫 綜整。這會使每個任務多出 2–3 個額外迴圈,並使延遲增加 40%。你的評估顯示,這些驗證中有 85% 是簡單的事實查核(日期、姓名、統計數字),15% 則需要更深入的研究。哪種做法能最有效地降低開銷,同時保有系統可靠性?

哪種做法最有效?

- A) 讓 綜整代理存取所有 web-search 工具,使其能直接處理任何驗證需求,而無需經過協調者迴圈。
- B) 讓 綜整代理累積所有驗證需求,並在最後以批次形式回傳給協調者,再由協調者一次將它們全部送給 web-search 代理。
- C) 讓 web-search 代理在初始研究期間,主動快取每個來源周邊的額外上下文,以預先因應 綜整 可能需要的驗證。
- D) 給 綜整代理一個範圍受限的 `verify_fact` 工具來進行簡單查核,同時將複雜的驗證透過協調者轉送給 web-search 代理。 **[CORRECT]**

為何選 D: 範圍受限的事實驗證工具讓 綜整代理能直接處理 85% 的簡單查核,消除了大多數迴圈,同時為那 15% 的複雜驗證保留協調者委派路徑。這套做法在大幅降低延遲的同時,落實了最小權限原則。

情境:用於持續整合的 Claude Code

問題 16(情境:用於持續整合的 Claude Code)

情境: 你的 CI 管線執行 Claude Code CLI(以 `--print` 模式),並使用 CLAUDE.md 為程式碼審查提供專案上下文,而開發者普遍認為這些審查很有實質內容。然而,他們回報將審查結果整合進工作流程很困難——Claude 輸出的是敘述性段落,必須手動複製到 PR 留言中。團隊希望能自動將每個發現以獨立的行內 PR 留言形式,張貼在程式碼中相關的位置,這需要包含檔案路徑、行號、嚴重程度與建議修正的結構化資料。哪種做法最有效?

哪種做法最有效?

- A) 在 CLAUDE.md 中新增一個「審查輸出格式」章節,並附上結構化發現的範例,讓 Claude 從專案上下文中學習預期的格式。
- B) 使用 CLI 旗標 `--output-format json` 與 `--json-schema` 來強制產生結構化發現,然後解析輸出,透過 GitHub API 張貼行內留言。 **[CORRECT]**
- C) 在審查提示中加入明確的格式化指令,要求每個發現遵循一個可解析的範本,例如 `[FILE:path] [LINE:n] [SEVERITY:level] ...`。
- D) 保留敘述性審查格式,但新增一個摘要步驟,使用 Claude 產生發現的結構化 JSON 摘要。

為何選 B: 使用 `--output-format json` 搭配 `--json-schema`,能在 CLI 層級強制結構化輸出,保證產生格式良好且具備所需欄位(檔案路徑、行號、嚴重程度、建議修正)的 JSON,可被可靠地解析並透過 GitHub API 張貼為行內 PR 留言。它善用了專為結構化輸出而設計的內建 CLI 功能。

問題 17(情境:用於持續整合的 Claude Code)

情境: 你的團隊使用 Claude Code 來產生程式碼建議,但你注意到一種模式:不明顯的問題——會破壞邊界案例的效能最佳化、會意外改變行為的清理動作——只有在另一位團隊成員審查 PR 時才會被發現。Claude 在產生過程中的推理顯示,它確實考慮過這些情況,卻得出自己的做法正確的結論。哪種做法能直接解決這種自我檢查侷限的根本原因?

哪種做法能直接解決根本原因?

- A) 執行第二個獨立的 Claude Code 實例來審查變更,且不讓它存取產生器的推理過程。 **[CORRECT]**
- B) 為產生階段啟用擴展思考模式,以便在產出建議前進行更周詳的審議。
- C) 在產生提示中加入明確的自我審查指令,要求 Claude 在定稿輸出前批評自己的建議。
- D) 在提示上下文中納入完整的測試檔案與文件,讓 Claude 在產生時更能理解預期行為。

為何選 A: 第二個獨立的 Claude Code 實例在不存取產生器推理過程的情況下,透過避免確認偏誤,直接解決了根本原因。這種「新視角」的觀點反映了人類同儕審查的精神,亦即由另一位審查者抓出作者已為其自圓其說的問題。

問題 18(情境:用於持續整合的 Claude Code)

情境: 你的程式碼審查元件是迭代式的:Claude 分析變更後的檔案,接著可能透過工具呼叫請求相關檔案(匯入、基底類別、測試),以在提供最終回饋前理解上下文。你的應用程式定義了一個工具,讓 Claude 能請求檔案內容;Claude 呼叫該工具、取得結果,然後繼續分析。你正在評估以批次處理來降低 API 成本。在考慮為此工作流程採用批次處理時,主要的技術限制是什麼?

主要的技術限制是什麼?

- A) 批次處理不包含關聯 ID,無法將輸出對應回輸入請求。
- B) 非同步模型無法在請求進行中執行工具並回傳結果,供 Claude 繼續分析。 **[CORRECT]**
- C) Batch API 不支援在請求參數中提供工具定義。
- D) 批次處理長達 24 小時的延遲對 pull request 回饋而言太慢,儘管該工作流程在其他方面仍可運作。

為何選 B: 「射後不理」式的非同步 Batch API 模型,沒有任何機制能在請求進行中攔截工具呼叫、執行工具,並回傳結果供 Claude 繼續分析。這與需要在單一邏輯互動中進行多輪工具請求/回應的迭代式工具呼叫工作流程根本上不相容。

問題 19(情境:用於持續整合的 Claude Code)

情境: 你的 CI/CD 系統執行三種以 Claude 為基礎的分析:(1) 在每個 PR 上進行快速風格檢查,並在完成前阻擋合併;(2) 對整個程式碼庫進行每週的全面安全稽核;(3) 對近期變更的模組進行每晚的測試案例產生。

Message Batches API 可節省 50% 成本,但處理可能長達 24 小時。你希望在維持可接受的開發者體驗的同時最佳化 API 成本。哪個組合正確地將每項任務對應到一種 API 做法?

哪個組合正確?

- A) 三項任務全部使用 Message Batches API 以最大化 50% 的節省,並設定管線輪詢批次完成狀態。
- B) PR 風格檢查使用同步呼叫;每週安全稽核與每晚測試產生使用 Message Batches API。 **[CORRECT]**
- C) 三項任務全部使用同步呼叫以獲得一致的回應時間,並依靠提示快取來降低各工作負載的成本。
- D) PR 風格檢查與每晚測試產生使用同步呼叫;只有每週安全稽核使用 Message Batches API。

為何選 B: PR 風格檢查會阻擋開發者,需要透過同步呼叫立即回應;而每週安全稽核與每晚測試產生是排程任務,期限有彈性,能容忍長達 24 小時的批次視窗——兩者皆可取得 50% 的節省。

問題 20(情境:用於持續整合的 Claude Code)

情境: 你的自動化審查找出了真正的問題,但開發者回報這些回饋不具可行動性。發現中包含像「複雜的工單路由邏輯」或「潛在的空指標」這類語句,卻沒有明確指出究竟該改什麼。當你加入詳細指令,例如「務必納入具體的修正建議」時,模型仍會產生不一致的輸出——時而詳細,時而含糊。哪種提示技巧最能可靠地產生一致且可行動的回饋?

哪種提示技巧最可靠?

- A) 進一步精修指令,對回饋格式的各部分(位置、問題、嚴重程度、建議修正)提出更明確的要求。
- B) 擴大上下文視窗以納入更多周邊程式碼庫,讓模型有足夠資訊提出具體修正。
- C) 實作兩遍式做法,由一個提示找出問題、第二個提示產生修正,讓兩者各司其職。
- D) 加入 3–4 個少樣本範例,展示確切要求的格式:找出的問題、程式碼中的位置、具體的修正建議。 **[CORRECT]**

為何選 D: 當僅靠指令會產生不一致結果時,少樣本範例是達成一致輸出格式最有效的技巧。提供 3–4 個展示確切預期結構(問題、位置、具體修正)的範例,能給模型一個具體的模式可循,這比抽象的指令更可靠。

問題 21(情境:用於持續整合的 Claude Code)

情境: 你的 CI 管線包含兩種以 Claude 為基礎的程式碼審查模式:一種是 pre-merge-commit hook,會在完成前阻擋 PR 合併;另一種是「深度分析」,於夜間執行、輪詢批次完成狀態,並將詳細建議張貼到 PR。你想透過 Message Batches API 來降低 API 成本,它提供 50% 的節省,但需要輪詢且最多可能耗時 24 小時。哪一種模式應該使用批次處理?

哪一種模式應該使用批次處理?

- A) 只有 pre-merge-commit hook。
- B) 只有深度分析。 **[CORRECT]**
- C) 兩種模式皆是。
- D) 兩種模式皆否。

為何選 B: 深度分析是批次處理的理想對象,因為它本來就在夜間執行、能容忍延遲,並在發佈結果前採用輪詢模型——這與 Message Batches API 那種非同步、以輪詢為基礎的架構相符,同時還能取得 50% 的節省。

問題 22(情境:用於持續整合的 Claude Code)

情境: 你的自動化審查會分析註解與 docstring。目前的提示指示 Claude「檢查註解是否準確且為最新狀態」。審查結果經常標記可接受的模式(TODO 標記、簡單描述),卻漏掉那些描述程式碼已不再實作之行為的註解。什麼變更能解決這種不一致分析的根本原因?

什麼變更能解決根本原因?

- A) 納入 `git blame` 資料,讓 Claude 能找出早於近期程式碼變更的註解。
- B) 加入具誤導性註解的少樣本範例,協助模型在程式碼庫中辨識類似的模式。
- C) 在分析前過濾掉 TODO、FIXME 與描述性註解模式,以減少雜訊。
- D) 指定明確的判定標準:只有當註解所宣稱的行為與程式碼實際行為矛盾時才標記。 **[CORRECT]**

為何選 D: 明確的判定標準——只有當註解所宣稱的行為與程式碼實際行為矛盾時才標記——以對「何謂問題」的精確定義取代模糊的指示,直接解決了根本原因。這能減少對可接受模式的誤報,以及對真正具誤導性註解的遺漏。

問題 23(情境:用於持續整合的 Claude Code)

情境: 你的自動化程式碼審查系統顯示出不一致的嚴重性評級——類似的問題(如 null pointer 風險)在某些 PR 被評為「critical」,在其他 PR 卻只評為「medium」。開發者問卷顯示信任度日漸下滑——許多人開始未讀就略過審查結果,因為「有一半是錯的」。高誤報的類別侵蝕了對準確類別的信任。哪種做法最能在改善系統的同時恢復開發者的信任?

哪種做法最能恢復開發者的信任?

- A) 暫時停用高誤報的類別(風格、命名、文件),只保留高精確度的類別,同時改善提示。 **[CORRECT]**
- B) 保持所有類別啟用,但在每項審查結果旁顯示信心分數,讓開發者自行決定要調查哪些。
- C) 保持所有類別啟用,並在接下來幾週加入少樣本範例,以改善每個類別的準確度。
- D) 對所有類別套用一致的嚴格度調降,以降低整體的誤報率。

為何選 A: 暫時停用高誤報的類別能立即阻止信任的侵蝕,移除那些導致開發者一概略過的雜訊審查結果,同時保留來自安全性與正確性等高精確度類別的價值。它也創造出空間,讓你在重新啟用前能先改善有問題類別的提示。

問題 24(情境:用於持續整合的 Claude Code)

情境: 你的自動化審查會為每個 PR 產生測試案例建議。在審查一個新增課程完成度追蹤的 PR 時,Claude 建議了 10 個測試案例,但開發者回饋顯示其中有 6 個重複了現有測試套件已涵蓋的情境。什麼變更最能有效減少重複的建議?

什麼變更最有效?

- A) 將現有的測試檔案納入上下文,讓 Claude 能判斷哪些情境已被涵蓋。 **[CORRECT]**
- B) 將請求的建議數量從 10 個減少到 5 個,假設 Claude 會優先處理最有價值的案例。

- C) 加入指示,引導 Claude 專注於邊界案例與錯誤情況,而非成功路徑。
- D) 實作後處理,透過關鍵字重疊過濾掉描述與現有測試名稱相符的建議。

為何選 A: 納入現有的測試檔案能修正重複的根本原因:唯有當 Claude 知道已存在哪些測試時,才能避免建議已被涵蓋的情境。這給了 Claude 提出真正全新、有價值測試所需的資訊。

問題 25(情境:用於持續整合的 Claude Code)

情境: 在一次初步自動化審查找出 12 項審查結果後,開發者推送了新的 commit 來處理這些問題。重新執行審查產生了 8 項審查結果,但開發者回報其中有 5 項重複了先前針對已在新 commit 中修正之程式碼的留言。在維持徹底性的同時,消除這種冗餘回饋最有效的方法是什麼?

消除冗餘回饋最有效的方法是什麼?

- A) 只在 PR 建立時與最終的 pre-merge 狀態執行審查,跳過中間的 commit。
- B) 加入後處理過濾,在張貼留言前,依檔案路徑與問題描述移除與先前相符的審查結果。
- C) 將審查範圍限制在最近一次推送所變更的檔案,排除較早 commit 的檔案。
- D) 將先前的審查結果納入上下文,並指示 Claude 只回報新的或仍未解決的問題。 **[CORRECT]**

為何選 D: 將先前的審查結果納入上下文,能讓 Claude 區分新問題與那些已在近期 commit 中處理過的問題。這在維持審查徹底性的同時,利用 Claude 的推理能力來避免對已修正程式碼提出冗餘的回饋。

問題 26(情境:用於持續整合的 Claude Code)

情境: 你的管線指令稿執行 `claude "Analyze this pull request for security issues"`,但作業卻無限期地卡住。日誌顯示 Claude Code 正在等待互動式輸入。在自動化管線中執行 Claude Code 的正確做法是什麼?

正確的做法是什麼?

- A) 加上 `--batch` 旗標:`claude --batch "Analyze this pull request for security issues"`。
- B) 加上 `-p` 旗標:`claude -p "Analyze this pull request for security issues"`。 **[CORRECT]**
- C) 將 stdin 從 `/dev/null` 重新導向:`claude "Analyze this pull request for security issues" < /dev/null`。
- D) 在執行指令前設定環境變數 `CLAUDE_HEADLESS=true`。

為何選 B: `-p`(或 `--print`)旗標是以非互動方式執行 Claude Code 的官方記載做法。它會處理提示、將結果印至 stdout,並在不等待使用者輸入的情況下結束——非常適合 CI/CD 管線。

問題 27(情境:用於持續整合的 Claude Code)

情境: 一個 pull request 變更了庫存追蹤模組中的 14 個檔案。將所有檔案一起分析的單遍審查產生了不一致的結果:對某些檔案有詳細回饋,對其他檔案卻只有膚淺的留言,漏掉了明顯的 bug,還出現矛盾的回饋(某個模式在一個檔案被標記,但同一個 PR 中另一個檔案的相同程式碼卻被核可)。你應該如何重新建構這個審查?

你應該如何重新建構這個審查?

- A) 執行三次獨立的全 PR 審查遍次,只標記在三次執行中至少出現兩次的問題。
- B) 拆分成專注的遍次:逐一審查每個檔案的局部問題,然後執行另一個以整合為導向的遍次,檢查跨檔案的資料流。 **[CORRECT]**
- C) 要求開發者在執行自動化審查前,先將大型 PR 拆分成 3~4 個檔案的較小提交。
- D) 改用具有更大上下文視窗的較大模型,讓它能在單一遍次中對全部 14 個檔案投入足夠的注意力。

為何選 B: 專注的逐檔案遍次透過確保一致的深度與可靠的局部問題偵測,解決了根本原因——注意力稀釋。接著,另一個以整合為導向的遍次則涵蓋跨檔案的考量,例如相依性與資料流的互動。

問題 28(情境:用於持續整合的 Claude Code)

情境: 你的自動化程式碼審查平均每個 pull request 產出 15 項發現,而開發人員回報有 40% 的誤報率。瓶頸在於調查時間:開發人員必須點進每一項發現,閱讀 Claude 的理由說明,才能決定要修正還是駁回。你的 CLAUDE.md 已經包含可接受模式的完整規則,而且利害關係人否決了任何在開發人員看到發現之前就先過濾的做法。哪一項變更最能解決調查時間的問題?

哪一項變更最能解決調查時間的問題?

- A) 要求 Claude 直接在每一項發現中附上其理由說明與信心估計值。**[CORRECT]**
- B) 加入一個後處理器,分析發現的模式,並自動抑制那些符合歷史誤報特徵的發現。
- C) 將發現分類為「阻擋性問題」與「建議」,並依層級設定不同的審查要求。
- D) 設定 Claude 只顯示高信心的發現,在開發人員看到之前先過濾掉不確定的標記。

為何選 A: 在每一項發現中直接附上理由說明與信心,可讓開發人員快速分流而不必逐項點開,進而減少調查時間。它滿足「不過濾」的限制條件,因為所有發現都仍然可見,同時加速了開發人員的決策。

問題 29(情境:用於持續整合的 Claude Code)

情境: 對你的自動化程式碼審查所做的分析顯示,不同發現類別之間的誤報率差異很大:安全性/正確性發現有 8% 誤報,效能發現 18%,風格/命名發現 52%,而文件發現 48%。開發人員調查顯示不信任感日益升高——許多人開始連看都不看就駁回發現,因為「有一半都是錯的」。高誤報的類別侵蝕了人們對準確類別的信任。哪一種做法最能恢復開發人員的信任,同時改善整個系統?

哪一種做法最能恢復開發人員的信任?

- A) 暫時停用高誤報類別(風格、命名、文件),只保留高精準度的類別,同時改善提示。**[CORRECT]**
- B) 保持所有類別啟用,但在每一項發現旁顯示信心分數,讓開發人員自行決定要調查哪些。
- C) 保持所有類別啟用,並在接下來幾週內加入少樣本範例,以提升各類別的準確度。
- D) 對所有類別套用一致的嚴格度調降,使整體誤報率下降。

為何選 A: 暫時停用高誤報類別,能立即透過移除那些導致開發人員一律駁回的雜訊發現,來阻止信任的侵蝕,同時保留安全性與正確性等高精準度類別所帶來的價值。它也為在重新啟用之前改善問題類別的提示創造了空間。

問題 30(情境:用於持續整合的 Claude Code)

情境: 你的團隊想要降低自動化分析的 API 成本。目前,同步的 Claude 呼叫支援兩種工作流程:(1) 一個阻擋式的合併前檢查,必須在開發人員能夠合併之前完成;以及 (2) 一份在夜間產生、供隔天早上審閱的技術債報告。你的經理提議將兩者都改用 Message Batches API 以節省 50%。你應該如何評估這項提議?

你應該如何評估這項提議?

- A) 將兩者都改用批次處理,並在批次耗時過久時回退到同步呼叫。
- B) 將兩種工作流程都改用批次處理,並以狀態輪詢來確認完成。
- C) 只對技術債報告使用批次處理;合併前檢查則保留同步呼叫。**[CORRECT]**
- D) 兩種工作流程都保留同步呼叫,以避免批次結果排序的問題。

為何選 C: Message Batches API 的處理最長可能需要 24 小時,且沒有延遲 SLA,這對夜間的技術債報告是可接受的,但對開發人員必須等待的阻擋式合併前檢查則無法接受。這做法依據延遲需求,將每種工作流程對應到正確的 API。

情境:使用 Claude Code 生成程式碼

問題 31(情境:使用 Claude Code 生成程式碼)

情境: 你要求 Claude Code 實作一個函式,將 API 回應轉換成內部的正規化格式。經過兩次迭代後,輸出結構仍然不符合預期——有些欄位的巢狀方式不同,而時間戳記的格式也不正確。你以散文方式描述了需求,但 Claude 每次的解讀都不一樣。

對於下一次迭代,哪一種做法最有效?

- A) 撰寫一份描述預期輸出結構的 JSON 結構描述,並在每次迭代後拿 Claude 的輸出來對它驗證。
- B) 提供 2 至 3 組具體的輸入-輸出範例,展示代表性 API 回應的預期轉換結果。**[CORRECT]**
- C) 以更高的技術精準度重寫需求,指定確切的欄位對應、巢狀規則與時間戳記格式字串。
- D) 請 Claude 說明它目前對需求的理解,以找出解讀分歧之處。

為何選 B: 具體的輸入-輸出範例,透過向 Claude 展示確切的預期轉換結果,消除了散文描述中固有的不明確性。這直接針對了根本原因——對文字需求的誤解——提供了關於欄位巢狀與時間戳記格式的明確模式。

問題 32(情境:使用 Claude Code 生成程式碼)

情境: 你需要新增 Slack 作為新的通知管道。現有的程式碼庫對於電子郵件、簡訊與推播管道,已有清楚且確立的模式。然而,Slack 的 API 提供了根本上不同的整合方式——incoming webhooks(簡單、單向)、bot tokens(支援送達確認與程式化控制),或 Slack Apps(雙向事件,需要工作區核准)。你的任務只說「新增 Slack 支援」,並未指定整合方式,也未要求送達追蹤之類的進階功能。

你應該如何著手這項任務?

- A) 以直接執行模式開始,使用 incoming webhooks 來比照現有的單向通知模式。
- B) 切換到規劃模式,探索各種整合選項與架構上的影響,然後在實作前提出建議。**[CORRECT]**
- C) 以直接執行模式開始,使用現有模式搭建一個 Slack 管道類別的骨架,將整合方式的決策延後。
- D) 以直接執行模式開始,採用 bot-token 的做法,以確保能夠進行送達確認。

為何選 B: Slack 整合有多種有效的做法,且各自的架構影響差異甚大,而需求又不明確。規劃模式讓你能夠評估 webhooks、bot tokens 與 Slack Apps 之間的權衡取捨,並在實作前對某個做法取得共識。

問題 33(情境:使用 Claude Code 生成程式碼)

情境: 你的 CLAUDE.md 檔案已增長到 400 多行,包含程式設計標準、測試慣例、一份詳細的 PR 審查檢查清單、部署指示,以及資料庫遷移程序。你希望 Claude 永遠遵守程式設計標準與測試慣例,但只在執行那些任務時才套用 PR 審查、部署與遷移的指引。

哪一種重新組織的做法最有效?

- A) 將所有指引移入依工作流程類型組織的獨立 Skills 檔案,在 CLAUDE.md 中只留下簡短的專案說明。
- B) 全部保留在 CLAUDE.md 中,但使用 `@import` 語法依類別組織成各自維護的檔案。
- C) 將 CLAUDE.md 拆分成 `.claude/rules/` 底下的檔案,並搭配路徑綁定的 glob 模式,讓每條規則只在相關的檔案類型才載入。
- D) 將通用標準保留在 CLAUDE.md 中,並為工作流程專屬的指引(PR 審查、部署、遷移)建立帶有觸發關鍵字的 Skills。**[CORRECT]**

為何選 D: CLAUDE.md 的內容會在每個工作階段載入,確保程式設計標準與測試慣例永遠適用;而 Skills 則是在 Claude 偵測到觸發關鍵字時才按需叫用——對於 PR 審查、部署與遷移之類的工作流程專屬指引而言,這是理想的做法。

問題 34(情境:使用 Claude Code 生成程式碼)

情境: 你被指派將團隊的單體式應用程式重新組織成微服務。這會影響數十個檔案的變更,並需要對服務邊界與模組相依性做出決策。

你應該選擇哪一種做法?

- A) 切換到規劃模式,探索程式碼庫、了解相依性,並在進行變更之前設計實作的做法。**[CORRECT]**
- B) 以直接執行模式開始,只在實作期間遇到非預期的複雜度時才切換到規劃。
- C) 以直接執行模式開始並做出漸進式變更,讓實作過程自然揭示出服務邊界。
- D) 使用直接執行,並搭配指定每個服務結構的詳細前置指示。

為何選 A: 對於像拆分單體應用這類複雜的架構重組,規劃模式是正確的策略:它讓你能在投入跨許多檔案、可能代價高昂的變更之前,安全地探索並對邊界做出明智的決策。

問題 35(情境:使用 Claude Code 生成程式碼)

情境: 你的團隊建立了一個 `/analyze-codebase` 技能,可執行深度程式碼分析——相依性掃描、測試涵蓋率計數,以及程式碼品質指標。執行該命令後,團隊成員回報 Claude 在工作階段中變得較不靈敏,並且遺失了原始任務的上下文。

在保留完整分析能力的前提下,你要如何最有效地修正這個問題?

- A) 在技能 frontmatter 中加入 `context: fork`,讓分析在隔離的子代理上下文中執行。**[CORRECT]**
- B) 在 frontmatter 中加入 `model: haiku`,使用更快、更便宜的模型進行分析。
- C) 將技能拆成三個較小的技能,各自產生較少的輸出。
- D) 在技能中加入指令,在顯示結果之前先將所有結果壓縮成簡短摘要。

為何選 A: `context: fork` 讓分析在隔離的子代理上下文中執行,因此大量輸出不會汙染主工作階段的上下文視窗,Claude 也不會遺失對原始任務的掌握。它在保留完整分析能力的同時,維持了主工作階段的靈敏度。

問題 36(情境:使用 Claude Code 生成程式碼)

情境: 你的團隊使用位於 `.claude/skills/commit/SKILL.md` 的 `/commit` 技能。某位開發者想為自己的個人工作流程進行客製化(不同的提交訊息格式、額外的檢查),且不影響其他隊友。

你會建議什麼做法?

- A) 在 `~/.claude/skills/` 下以不同名稱建立個人版本,例如 `/my-commit`。
- B) 在專案技能 frontmatter 中加入以使用者名稱為條件的判斷邏輯。
- C) 在 `~/.claude/skills/commit/SKILL.md` 以相同名稱建立個人版本。**[CORRECT]**
- D) 在個人技能 frontmatter 中設定 `override: true`,使其優先於專案版本。

為何選 C: 技能的解析取決於存放位置:企業層級覆蓋個人層級,個人層級覆蓋專案層級。因此放在 `~/.claude/skills/commit/SKILL.md` 的個人技能,只會對這位開發者遮蔽團隊的 `.claude/skills/commit/SKILL.md`——儲存庫中沒有任何變動,隊友仍然執行專案版本,`/commit` 對其他所有人的行為完全不變。選項 A 在機制上可行,但作為建議較弱:專案的 `/commit` 會與 `/my-commit` 並存,而兩個 `description` 幾乎相同的技能會讓 Claude 的自動技能選擇變得難以預測,同時也破壞了「執行 `/commit`」這個使用習慣與團隊文件。選項 B 不成立,因為技能 frontmatter 沒有條件判斷或使用者名稱的機制,而且這麼做需要修改題目已排除的共用專案檔案。選項 D 不成立,因為根本沒有 `override` 這個欄位——優先順序來自存放位置,而非宣告。

問題 37(情境:使用 Claude Code 生成程式碼)

情境: 你的團隊已使用 Claude Code 數個月。最近,有三位開發者回報 Claude 會遵循「一律包含完整的錯誤處理」這項指引,但一位剛加入的第四位開發者表示 Claude 並未遵循。四人都在同一個儲存庫中工作,且程式碼皆為最新。

最可能的原因與修正方式是什麼?

- A) 該指引存在於原本那幾位開發者的使用者層級 `~/.claude/CLAUDE.md` 檔案中,而非專案的 `.claude/CLAUDE.md`。將該指令移至專案層級的檔案,讓所有團隊成員都能收到。**[CORRECT]**
- B) 新開發者的 `~/.claude/CLAUDE.md` 包含相衝突的指令,覆寫了專案設定;他們應刪除衝突的段落。
- C) Claude Code 會隨時間學習個別使用者的偏好;新開發者必須重複該需求,直到 Claude「記住」為止。
- D) Claude Code 在首次讀取後會快取 CLAUDE.md;原本的開發者使用的是快取版本。每個人都應清除 Claude Code 的快取。

為何選 A: 如果該指引只被加入原本那幾位開發者的使用者層級設定,而未加入專案層級的 `.claude/CLAUDE.md`,新團隊成員就不會收到。將它移至專案層級的設定,可確保所有現有與未來的團隊成員自動取得該指引。

問題 38(情境:使用 Claude Code 生成程式碼)

情境: 你發現,在生成新的 API 端點時,將 2–3 個完整的端點實作範例作為上下文納入,可顯著提升一致性。然而,這個上下文只有在建立新端點時才有用——在除錯、審查程式碼或於 API 目錄中進行其他工作時則無用。

哪一種設定方式最為有效?

- A) 將端點範例與模式文件加入專案 CLAUDE.md,使其隨時可用。
- B) 在每次生成請求中,以複製程式碼到提示的方式手動引用端點範例。
- C) 在 `.claude/rules/api/` 中設定路徑專屬規則,納入端點範例,並在 API 目錄中工作時啟用。
- D) 建立一個引用端點範例並包含模式遵循指令的技能,透過斜線命令按需呼叫。**[CORRECT]**

為何選 D: 按需呼叫的技能只會在生成新端點時載入範例上下文,而不會在除錯或審查等不相關的任務期間載入。這讓主上下文保持乾淨,同時在需要時保留高品質的生成能力。

問題 39(情境:使用 Claude Code 生成程式碼)

情境: 你的團隊建立了一個 `/migration` 技能,用來生成資料庫遷移檔案。它透過 `$ARGUMENTS` 取得遷移名稱。在正式環境中你觀察到三個問題:(1) 開發者經常在未提供引數的情況下執行該技能,導致檔案命名不佳;(2) 該技能有時會使用來自不相關的先前對話中的資料庫結構描述細節;(3) 某位開發者在該技能擁有廣泛工具存取權時,意外執行了破壞性的測試清理。

哪一種設定方式能修正這三個問題?

- A) 使用位置參數 `$1` 與 `$2` 取代 `$ARGUMENTS` 以強制特定輸入,透過 `@` 語法明確引用結構描述檔案以控制上下文,並在 frontmatter 描述中加入關於破壞性操作的警告。
- B) 在 frontmatter 中加入 `argument-hint` 以要求必填參數,使用 `context: fork` 來隔離執行,並將 `allowed-tools` 限制為檔案寫入操作。**[CORRECT]**
- C) 拆成 `/migration-create` 與 `/migration-apply` 兩個技能,加入驗證指令以在缺少遷移名稱時加以索取,並為各自使用不同的 `allowed-tools` 範圍。
- D) 在技能的 SKILL.md 中加入驗證指令以確保 `$ARGUMENTS` 是有效的名稱,加入提示以忽略先前的對話上下文,並列出應避免的禁止操作。

為何選 B: 這運用三項各自獨立的設定功能來分別解決每個問題:`argument-hint` 改善引數輸入並減少缺少引數的情況,`context: fork` 防止來自先前對話的上下文外洩,而 `allowed-tools` 將技能限制在安全的檔案寫入操作,從而防止破壞性動作。

問題 40(情境:使用 Claude Code 生成程式碼)

情境: 你的程式碼庫包含採用不同程式碼慣例的區域:React 元件使用搭配 hooks 的函式式風格,API 處理常式使用 async/await 搭配特定的錯誤處理,而資料庫模型則遵循 repository 模式。測試檔案分散在程式碼庫各處,緊鄰其受測程式碼(例如 `Button.test.tsx` 位於 `Button.tsx` 旁邊),而你希望所有測試無論位置為何都遵循相同的慣例。

要確保 Claude 在生成程式碼時自動套用正確的慣例,最受支援的方式是什麼?

- A) 將所有慣例放入根目錄的 CLAUDE.md,各區域各用一個標題,並依賴 Claude 自行推斷適用哪個段落。
- B) 在 `.claude/skills/` 中為每種程式碼類型建立技能,將慣例嵌入各個 SKILL.md。
- C) 在每個子目錄中放置一個獨立的 CLAUDE.md 檔案,內含該區域的慣例。
- D) 在 `.claude/rules/` 下建立規則檔案,以 YAML frontmatter 指定 glob 模式,依檔案路徑有條件地套用慣例。**[CORRECT]**

為何選 D: 帶有 YAML frontmatter 與 glob 模式(例如 `**/*.test.tsx`、`src/api/**/*.ts`)的 `.claude/rules/` 檔案,能無論目錄結構為何都實現確定性、以路徑為基礎的慣例套用。對於像分散的測試檔案這類橫切模式,這是最受支援的做法。

問題 41(情境:使用 Claude Code 生成程式碼)

情境: 你想建立一個自訂斜線命令 `/review`,用來執行你團隊的標準程式碼審查檢查清單。它應在每位開發者複製或更新儲存庫時都可使用。

你應該在哪裡建立這個命令檔案?

- A) 在每位開發者家目錄中的 `~/.claude/commands/`。
- B) 在專案儲存庫中的 `.claude/commands/` 下。**[CORRECT]**
- C) 在 `.claude/config.json` 中作為一個命令陣列。
- D) 在根目錄的專案 CLAUDE.md 中。

為何選 B: 將自訂斜線命令放在專案儲存庫內的 `.claude/commands/` 下,可確保它們納入版本控制,並對每位複製或更新儲存庫的開發者自動可用。這是 Claude Code 中專案層級自訂命令的預期存放位置。

問題 42(情境:使用 Claude Code 生成程式碼)

情境: 你的團隊 CLAUDE.md 已成長到超過 500 行,混雜了 TypeScript 慣例、測試指引、API 模式與部署程序。開發人員難以找到並更新正確的章節。

Claude Code 支援哪種做法,將專案層級的指令整理成聚焦的主題模組?

- A) 定義一個 `.claude/config.yaml` 對應檔,將檔案模式對應到 CLAUDE.md 內的特定章節。
- B) 在 `.claude/rules/` 中建立各自獨立的 Markdown 檔案,每個檔案涵蓋一個主題(例如 `testing.md`、`api-conventions.md`)。**[CORRECT]**
- C) 將指令拆分到相關子目錄下的 README.md 檔案,Claude 會自動將其載入為指令。
- D) 在目錄樹的不同層級建立多個名為 CLAUDE.md 的檔案,每個都覆寫上層指令。

為何選 B: Claude Code 支援一個 `.claude/rules/` 目錄,你可以在其中為主題指引建立各自獨立的 Markdown 檔案(例如 `testing.md`、`api-conventions.md`),讓團隊能將龐大的指令集整理成聚焦、易於維護的模組。

問題 43(情境:使用 Claude Code 生成程式碼)

情境: 你建立了一個自訂技能 `/explore-alternatives`,你的團隊用它在選定方案前先腦力激盪並評估各種實作方法。開發人員回報,執行該技能後,Claude 後續的回應會受到方案討論的影響——有時會引用已被否決的方法,或保留干擾實際實作的探索上下文。

你應如何最有效地設定這個技能?

- A) 在技能中使用 `!` 前綴,將探索邏輯當作 bash 子程序執行。
- B) 在技能的 frontmatter 中加入 `context: fork`。**[CORRECT]**
- C) 拆分成兩個技能——`/explore-start` 與 `/explore-end`——用以標示何時應捨棄探索上下文的邊界。
- D) 在 `~/.claude/skills/` 而非 `.claude/skills/` 中建立該技能。

為何選 B: `context: fork` 會在隔離的子代理上下文中執行該技能,使探索討論不會污染主對話歷史。這可避免已被否決的方法與腦力激盪的上下文影響後續的實作工作。

問題 44(情境:使用 Claude Code 生成程式碼)

情境: 你的團隊想透過 Claude Code 新增一個 GitHub MCP 伺服器,用於搜尋 PR 與檢查 CI 狀態。六位開發人員各自擁有自己的個人 GitHub 存取 token。你希望團隊間擁有一致的工具,同時不將憑證提交到版本控制中。

哪種設定做法最有效?

- A) 讓每位開發人員透過 `claude mcp add --scope user` 在使用者範圍中新增該伺服器。
- B) 建立一個 MCP 伺服器包裝層,從 `.env` 檔案讀取 token 並代理 GitHub API 呼叫,然後將該包裝層加入專案的 `.mcp.json`。
- C) 將伺服器加入專案的 `.mcp.json`,使用環境變數替換(`${GITHUB_TOKEN}`)進行驗證,並在專案 README 中說明所需的環境變數。**[CORRECT]**
- D) 在專案範圍中以佔位 token 設定伺服器,然後告知開發人員在其本機設定中覆寫它。

為何選 C: 採用環境變數替換的專案 `.mcp.json` 是慣用做法:它為 MCP 設定提供單一、受版本控制的真實來源,同時讓每位開發人員透過環境變數提供憑證。說明該變數可讓新人上手變得容易,而無須提交機密。

問題 45(情境:使用 Claude Code 生成程式碼)

情境: 你正在一個 120 個檔案的程式碼庫中,為外部 API 呼叫加上錯誤處理包裝層。這項工作分為三個階段:(1)找出所有呼叫點與模式,(2)協同設計錯誤處理方法,(3)一致地實作包裝層。在第 1 階段,Claude 產生了大量輸出,列出數百個帶有上下文的呼叫點,在探索完成前就迅速填滿了上下文視窗。

哪種做法最能有效完成任務,同時維持實作一致性?

- A) 在第 1 階段使用 Explore 子代理,以隔離冗長的探索輸出並回傳摘要,然後在主對話中繼續進行第 2–3 階段。**[CORRECT]**
- B) 在主對話中執行所有階段,逐步處理檔案時定期使用 `/compact` 來降低上下文用量。
- C) 切換到無頭模式並使用 `--continue`,在批次呼叫之間傳遞明確的上下文摘要以維持連續性。
- D) 在 CLAUDE.md 中定義錯誤處理模式,然後跨多個工作階段分批處理檔案,依賴共用的記憶檔案來維持一致性。

為何選 A: Explore 子代理會將冗長的探索輸出隔離在獨立的上下文中,只回傳精簡的摘要給主對話。這保留了主上下文視窗,供協同設計與一致實作這兩個最需要保留上下文的階段使用。

情境:客戶支援代理

問題 46(情境:客戶支援代理)

情境: 在測試時,你注意到當使用者詢問訂單狀態時,代理經常呼叫 `get_customer`,即使 `lookup_order` 會更適合。為了解決這個問題,你應該先檢查什麼?

你應該先檢查什麼?

- A) 實作一個前置處理分類器,以偵測與訂單相關的請求並將其直接路由到 `lookup_order`。
- B) 減少代理可用的工具數量,以簡化選擇。
- C) 在系統提示中加入少樣本範例,涵蓋所有可能的訂單請求模式,以改善工具選擇。
- D) 檢查工具描述,確保它們清楚區分每個工具的用途。**[CORRECT]**

為何選 D: 工具描述是模型用來決定要呼叫哪個工具的主要輸入。當代理持續挑選錯誤的工具時,第一個診斷步驟就是驗證工具描述是否清楚劃分每個工具的用途與使用界線。

問題 47(情境:客戶支援代理)

情境: 你的代理在處理單一問題的請求時準確率達 94%(例如「我需要為訂單 #1234 退款」)。但當客戶在一則訊息中包含多個問題時(例如「我需要為訂單 #1234 退款,同時也想更新訂單 #5678 的寄送地址」),工具選擇準確率降至 58%。代理通常只解決一個問題,或在不同請求間混用參數。哪種做法最能有效提升多問題請求的可靠性?

哪種做法最有效?

- A) 實作一個前置處理層,使用另一次獨立的模型呼叫,將多問題訊息分解為各自獨立的請求,各自獨立處理後再合併結果。
- B) 將相關的工具合併為較少的通用工具。
- C) 在提示中加入少樣本範例,示範多問題請求的正確推理與工具呼叫順序。**[CORRECT]**
- D) 實作回應驗證,偵測不完整的答案並自動重新提示代理以解決遺漏的問題。

為何選 C: 示範多問題請求正確推理與工具呼叫順序的少樣本範例最為有效,因為代理在單一問題上已表現良好——它所需要的是分解並路由多個問題、並保持參數分離的模式指引。

問題 48(情境:客戶支援代理)

情境: 正式環境日誌顯示,對於像「為訂單 #1234 退款」這類簡單請求,你的代理以 3–4 次工具呼叫解決問題,成功率達 91%。但對於像「我被重複扣款、我的折扣沒有套用,而且我想取消」這類複雜請求,代理平均需要 12 次以上的工具呼叫,成功率僅 54%——往往是逐一依序調查各問題,並為每個問題重複擷取相同的客戶資料。哪種變更最能有效改善複雜請求的處理?

哪種變更最有效?

- A) 在各階段之間加入明確的驗證檢查點,要求代理在解決每個問題後先記錄進度,再進行下一個。
- B) 透過將 `get_customer`、`lookup_order` 與計費相關工具合併為單一的 `investigate_issue` 工具,減少工具數量。
- C) 將請求分解為各自獨立的問題,然後利用共用的客戶上下文平行調查每一個,再綜合出最終的解決方案。**[CORRECT]**
- D) 在系統提示中加入少樣本範例,示範各種多面向計費情境的理想工具呼叫順序。

為何選 C: 分解為各自獨立的問題並以共用客戶上下文平行調查,能修正兩個關鍵問題:它透過在各問題間重複使用共用上下文,消除了重複的資料擷取;並透過在綜合單一解決方案前平行化調查,減少了工具呼叫的總迴圈次數。

問題 49(情境:客戶支援代理)

情境: 你的代理達到 55% 的首次接觸解決率,遠低於 80% 的目標。日誌顯示它把簡單案例(附照片佐證的受損商品標準換貨)升級,卻試圖自主處理需要政策例外的複雜情況。改善升級校準最有效的方式是什麼?

改善升級校準最有效的方式是什麼?

- A) 要求代理在每次回應前以 1–10 分自評信心,並在信心低於某個門檻時自動轉交真人。
- B) 部署一個以歷史工單訓練的獨立分類器模型,在主代理開始處理前先預測哪些請求需要升級。
- C) 在系統提示中加入明確的升級準則,並附上少樣本範例,說明何時該升級、何時該自主解決。**[CORRECT]**
- D) 實作情緒分析以判斷客戶的不滿程度,並在超過某個負面情緒門檻時自動升級。

為何選 C: 帶有少樣本範例的明確升級準則直接針對根本原因——簡單與複雜案例之間不清楚的決策邊界。它是最相稱、最有效的首要介入手段,能在不增加額外基礎設施的情況下,教會代理何時該升級、何時該自主

解決。

問題 50(情境:客戶支援代理)

情境: 在呼叫 `get_customer` 與 `lookup_order` 之後,代理已取得所有可用的系統資料,但仍面臨不確定性。哪一種情況最有正當理由觸發呼叫 `escalate_to_human`?

哪一種情況最有正當理由升級?

- A) 客戶想取消一筆昨天出貨、明天就會送達的訂單。代理應該升級,因為客戶在收到包裹後可能會改變心意。
- B) 客戶聲稱自己沒有收到訂單,但追蹤紀錄顯示三天前已送達其地址並有簽收。代理應該升級,因為提出相互矛盾的證據可能損害客戶關係。
- C) 客戶要求比照競爭對手的價格。你的政策允許在 14 天內針對自家網站的降價做價格調整,但對競爭對手的價格隻字未提。代理應該升級以進行政策解讀。**[CORRECT]**
- D) 客戶訊息同時包含一個帳單問題與一筆商品退貨。代理應該升級,讓真人在一次互動中協調這兩個議題。

為何選 C: 這是一個真正的政策缺口:公司規則涵蓋自家網站的降價,但並未處理比照競爭對手價格的情形。代理絕不能自行發明政策,而應升級,讓真人判斷如何解讀或延伸既有規則。

問題 51(情境:客戶支援代理)

情境: 正式環境日誌顯示,在 12% 的案例中,你的代理跳過 `get_customer`,僅憑客戶提供的姓名直接呼叫 `lookup_order`,有時導致帳戶辨識錯誤與退款錯誤。哪一項變更最能有效修正這個可靠性問題?

哪一項變更最有效?

- A) 加入少樣本範例,示範代理一律先呼叫 `get_customer`,即使客戶主動提供了訂單細節也一樣。
- B) 實作一個路由分類器,分析每個請求,並只啟用適合該請求類型的工具子集。
- C) 加入一個程式化的前置條件,在 `get_customer` 回傳經驗證的客戶識別碼之前,阻擋 `lookup_order` 與 `process_refund`。**[CORRECT]**
- D) 強化系統提示,聲明在任何訂單操作之前都必須透過 `get_customer` 進行客戶驗證。

為何選 C: 程式化的前置條件提供了確定性的保證,確保遵循必要的執行順序。它是最有效的做法,因為無論 LLM 行為如何,都能消除跳過驗證的可能性。

問題 52(情境:客戶支援代理)

情境: 正式環境指標顯示,在處理複雜的帳單爭議或多筆訂單退貨時,客戶滿意度分數比簡單案例低 15%——即使解決方案在技術上是正確的。根因分析顯示代理提供了正確的解決方案,但在說明理由時前後不一致:有時省略相關的政策細節,有時遺漏時程資訊或後續步驟。具體的上下文缺口因案例而異。你想在不增加人工監督的前提下改善解決方案品質。哪一種做法最有效?

哪一種做法最有效?

- A) 加入一個自我批判階段,讓代理評估草稿回應的完整性——確保它解決了客戶的問題、包含相關上下文,並預先設想後續問題。**[CORRECT]**
- B) 加入一個確認階段,讓代理在結案前詢問「這是否已完全解決你的問題?」,讓客戶在需要時可以要求額外資訊。
- C) 針對複雜案例,將模型從 Haiku 升級到 Sonnet,並依據定義好的複雜度指標進行路由。
- D) 在系統提示中實作少樣本範例,針對五種常見的複雜案例類型展示完整說明,示範如何納入政策上下文、時程與後續步驟。

為何選 A: 自我批判階段(評估者—最佳化者模式)直接針對說明完整性不一致的問題,強迫代理在呈現草稿前,依照具體準則——例如政策上下文、時程與後續步驟——評估自己的草稿。這能在不需人工監督的情況下,捕捉到因案例而異的缺口。

問題 53(情境:客戶支援代理)

情境: 正式環境指標顯示你的代理平均每次解決需要 4 次以上的 API 迴圈。分析發現,即使一開始就同時需要 `get_customer` 與 `lookup_order`,Claude 往往仍在分開的連續輪次中分別請求這兩者。減少迴圈數量最有效的方式是什麼?

減少迴圈最有效的方式是什麼?

- A) 實作推測性執行,自動與任何被請求的工具平行呼叫可能需要的工具,並回傳所有結果,無論實際請求的是什麼。
- B) 提高 `max_tokens`,給 Claude 更多空間進行規劃並自然地合併工具請求。
- C) 建立像 `get_customer_with_orders` 這樣的複合工具,將常見的查詢組合捆綁成單次呼叫。
- D) 在提示中指示 Claude 把工具請求捆綁進單一輪次,並在下一次 API 呼叫前一併回傳所有結果。**[CORRECT]**

為何選 D: 提示 Claude 將相關的工具請求捆綁進單一輪次,善用了它一次請求多個工具的原生能力。它以最小的架構變更直接修正了連續呼叫的模式。

問題 54(情境:客戶支援代理)

情境: 正式環境日誌顯示一種模式:客戶引用特定金額(例如「我先前提到的 15% 折扣」),但代理卻以錯誤的數值回應。調查發現這些細節是在 20 多輪以前提到的,並被濃縮成像「曾討論過促銷定價」這類含糊的摘要。哪一項修正最有效?

哪一項修正最有效?

- A) 將摘要門檻從 70% 提高到 85%,讓對話在觸發摘要之前有更多空間。
- B) 將完整的對話歷史儲存在外部儲存中,並在代理偵測到像「如我先前所提」這類引用時實作檢索。
- C) 將交易性事實(金額、日期、訂單編號)擷取到一個持續性的「案例事實」區塊,於每則提示中納入,且置於摘要過的歷史之外。**[CORRECT]**
- D) 修改摘要提示,明確要求逐字保留所有數字、百分比、日期與客戶陳述的期望。

為何選 C: 摘要本質上會遺失精確的細節。將交易性事實擷取到摘要過的歷史之外的一個結構化「案例事實」區塊,能保留關鍵資訊,使其無論已摘要了多少輪次,都能在每則提示中可靠地取用。

問題 55(情境:客戶支援代理)

情境: 你的 `get_customer` 工具在以姓名搜尋時會回傳所有相符項目。目前當有多筆結果時,Claude 會挑選最近有訂單的客戶,但正式環境資料顯示,對於不明確的相符項目,這種做法有 15% 的機率選錯帳戶。你該如何處理這個問題?

你該如何處理這個問題?

- A) 實作一套信心評分系統,在信心高於 85% 時自主行動,低於門檻時則要求澄清。
- B) 指示 Claude 在 `get_customer` 回傳多筆相符項目時,於採取任何客戶專屬動作之前,先要求一個額外的識別資訊(電子郵件、電話或訂單編號)。**[CORRECT]**
- C) 修改 `get_customer`,使其依據排序演算法只回傳單一最可能的相符項目,藉此消除不明確性。
- D) 在提示中加入少樣本範例,針對不明確的相符項目示範正確的推理與工具排序。

為何選 B: 向使用者詢問額外的識別資訊是化解不明確性最可靠的方式,因為使用者對自己的身分擁有確定的認知。多一次對話輪次,是消除因選錯帳戶而造成的 15% 錯誤率的微小代價。

問題 56(情境:客戶支援代理)

情境: 正式環境日誌顯示一個固定模式:當客戶在訊息中包含「account」這個字（例如「I want to check my account for an order I made yesterday」）時,代理有 78% 的機率會先呼叫 `get_customer`。當客戶以不含「account」的方式提出類似請求(例如「I want to check an order I made yesterday」)時,則有 93% 的機率會先呼叫 `lookup_order`。工具描述清楚且毫不含糊。這個差異最可能的根本原因是什麼?

最可能的根本原因是什麼?

- A) 系統提示包含對關鍵字敏感的指令,會根據「account」之類的詞彙引導行為,造成非預期的工具選擇模式。 **[CORRECT]**
- B) 模型的基礎訓練在「account」術語與客戶相關操作之間建立了關聯,凌駕於工具描述之上。
- C) 模型需要更多關於多概念訊息的訓練資料,並應以同時包含 account 與 order 術語的範例進行微調。
- D) 工具描述需要額外的負面範例,明確指出何時「不該」使用各工具,以防止這種由關鍵字引發的混淆。

為何選 A: 這種有系統、由關鍵字驅動的模式(78% 對 93%)強烈顯示系統提示中存在明確的路由邏輯,會對「account」這個字產生反應並將代理引導至客戶相關工具。由於工具描述本就清楚,這個差異指向提示層級的指令造成了非預期的行為引導。

問題 57(情境:客戶支援代理)

情境: 正式環境日誌顯示,當使用者詢問訂單時(例如「check my order #12345」),代理經常呼叫 `get_customer` 而非呼叫 `lookup_order`。兩個工具都只有極簡描述(「Gets customer information」/「Gets order details」),且接受外觀相似的識別碼格式。要提升工具選擇可靠性,最有效的第一步是什麼?

最有效的第一步是什麼?

- A) 實作一個路由層,在每一輪之前分析使用者輸入,並根據偵測到的關鍵字與 ID 模式預先選定正確的工具。
- B) 將兩個工具合併為單一的 `lookup_entity`,接受任何識別碼並在內部決定要查詢哪個後端。
- C) 在系統提示中加入少樣本範例以示範正確的工具選擇模式,以 5–8 個範例將訂單相關查詢路由至 `lookup_order`。
- D) 擴充每個工具的描述,納入輸入格式、範例查詢、邊界案例,以及說明何時該使用它而非類似工具的界線。 **[CORRECT]**

為何選 D: 以輸入格式、範例查詢、邊界案例與清楚界線來擴充工具描述,直接修正了根本原因——極簡的描述無法提供足夠資訊讓 LLM 區分相似的工具。這是一個低投入、高影響的第一步,能改善 LLM 用於工具選擇的主要機制。

問題 58(情境:客戶支援代理)

情境: 你正在為你的支援代理實作代理迴圈。在每次 Claude API 呼叫之後,你必須決定要繼續迴圈(執行所請求的工具並再次呼叫 Claude)或停止(向客戶呈現最終答案)。是什麼決定了這個判斷?

是什麼決定了這個判斷?

- A) 檢查 Claude 回應中的 `stop_reason` 欄位——若為 `tool_use` 則繼續,若為 `end_turn` 則停止。 **[CORRECT]**
- B) 解析 Claude 的文字,尋找「I'm done」或「Can I help with anything else?」之類的片語——自然語言訊號代表任務完成。
- C) 設定最大迭代次數(例如 10 次呼叫),達到上限即停止,無論 Claude 是否表示還需要更多工作。
- D) 檢查回應是否包含 assistant 文字內容——若 Claude 產生了說明性文字,迴圈就應該終止。

為何選 A: `stop_reason` 是 Claude 用於迴圈控制的明確結構化訊號:`tool_use` 表示 Claude 想執行工具並取回結果,而 `end_turn` 表示 Claude 已完成其回應,迴圈應該結束。

問題 59(情境:客戶支援代理)

情境: 正式環境日誌顯示代理會誤解你的 MCP 工具輸出:來自 `get_customer` 的 Unix 時間戳記、來自 `lookup_order` 的 ISO 8601 日期,以及數字狀態碼(1=pending、2=shipped)。有些工具是你無法修改的第三方 MCP 伺服器。哪一種資料格式正規化的方法最易於維護?

哪一種方法最易於維護?

- A) 使用 PostToolUse hook 攔截工具輸出,並在代理處理之前套用格式轉換。 **[CORRECT]**
- B) 修改你能控制的工具使其回傳人類可讀的格式,並為第三方工具建立包裝層。
- C) 建立一個 `normalize_data` 工具,讓代理在每次資料擷取後呼叫它來轉換數值。
- D) 在系統提示中加入詳細的格式文件,說明每個工具的資料慣例。

為何選 A: PostToolUse hook 提供一個集中、確定性的攔截點,可在代理處理之前正規化所有工具輸出——包括第三方 MCP 伺服器的資料。它更易於維護,因為轉換存在於程式碼中並一致地套用,而非仰賴 LLM 的解

讀。

問題 60(情境:客戶支援代理)

情境: 正式環境日誌顯示,代理有時會在 `lookup_order` 更為合適時選擇 `get_customer`,尤其是在「I need help with my recent purchase.」這類模糊的查詢上。你決定在系統提示中加入少樣本範例以改善工具選擇。哪一種方法最能有效解決問題?

哪一種方法最有效?

- A) 在每個工具描述中加入明確的「use when」與「don't use when」指引,涵蓋模糊的情況。
- B) 加入依工具分組的範例——所有 `get_customer` 情境放在一起,接著是所有 `lookup_order` 情境。
- C) 加入 4–6 個針對模糊情境的範例,每個都附上為何選擇某工具而非其他合理替代方案的理由。 **[CORRECT]**
- D) 加入 10–15 個清楚、毫不含糊的請求範例,為每個工具示範典型情境下的正確工具選擇。

為何選 C: 將少樣本範例針對實際發生錯誤的特定模糊情境,並明確說明為何某工具優於替代方案,能教會模型在邊界案例中所需的比較式決策過程。這比通用範例或宣告式規則更為有效。

問題 61(情境:對話式 AI 架構模式)

情境: 你的 `remove_team_member` 工具使用 `dry_run: boolean` 參數,在執行前預覽影響。正式環境監控顯示代理會直接以 `dry_run=false` 呼叫,藉此略過預覽步驟。你需要確保每一次移除都先經過使用者明確確認的預覽。

最可靠的方法是什麼?

- A) 加入伺服器端驗證,僅當過去 60 秒內曾發生過具有相同參數的 `dry_run=true` 呼叫時,才允許 `dry_run=false`。
- B) 將該工具標註為需要確認,並設定編排層,在將任何呼叫轉送至被標註的工具之前,先提示使用者核准。
- C) 在工具描述中加入詳細指令與少樣本範例,要求代理一律先以 `dry_run=true` 呼叫,並等待使用者確認後才再次呼叫。
- D) 改用兩個工具:`preview_remove_member` 回傳影響細節與一個一次性的確認 token;`execute_remove_member` 則需要該 token,將執行綁定至預覽。 **[CORRECT]**

為何選 D: 兩工具的 token 綁定方法在架構上讓人無法在未先預覽的情況下執行——execute 工具確實需要一個只有 preview 工具才能產生的 token。這是唯一在程式碼層級強制執行此約束的方法,而非仰賴 LLM 遵守指令(C)、時間啟發式(A)或編排基礎設施(B)。

問題 62(情境:對話式 AI 架構模式)

情境: 正式環境監控顯示你的 `search_catalog` 工具有 12% 的時間會失敗:其中 8% 是重試後會成功的網路逾時,4% 則是無論如何重試都不會成功的查詢語法錯誤。目前兩種錯誤類型的回傳方式完全相同,導致無謂的重試。

你應該如何修改該工具的錯誤處理?

- A) 在你的系統提示中加入少樣本範例,示範如何區分網路錯誤與語法錯誤。
- B) 對所有錯誤一致地套用指數退避重試邏輯。
- C) 在工具內部對網路逾時實作帶退避的自動重試;對語法錯誤則立即回傳並附上參數驗證細節。 **[CORRECT]**
- D) 為所有錯誤回傳一個 `retryable` 布林旗標與錯誤類型細節。

為何選 C: 在工具層級處理暫時性錯誤的重試是正確的抽象界線——工具對錯誤類型有確切的認知,且能實作確定性的重試邏輯,而不必仰賴代理解讀旗標(D)或遵循提示層級的指令(A)。一致的退避(B)會在永遠不會成功的語法錯誤上浪費時間。

問題 63(情境:對話式 AI 架構模式)

情境: 在討論投資策略的數輪對話中,某位使用者先表示「我的風險承受度非常低」,後來又說「我想要讓報酬最大化」。他們現在問:「我應該投資什麼?」

哪種做法最能確保建議符合使用者真正的優先考量?

- A) 點出這項矛盾,並請使用者釐清何者更重要。 **[CORRECT]**
- B) 針對兩種情境分別提供建議。
- C) 以最近一次陳述的偏好繼續進行。
- D) 在不處理衝突的情況下,推薦一個平衡型投資組合。

為何選 A: 當使用者偏好彼此直接矛盾時,點出衝突並請求釐清是唯一能保證建議符合使用者真實意圖的方式。任何其他做法都涉及一項可能錯誤的假設——報酬最大化與低風險承受度是本質上不相容的目標,需要由真人做出決定。

問題 64(情境:對話式 AI 架構模式)

情境: 使用者在多輪對話中逐步調整播放清單偏好。在某位使用者說「我愛爵士樂」後的兩則訊息,Claude 卻問「你喜歡哪些曲風?」

最可能的原因為何?

- A) Claude 需要連接向量資料庫才能維持對話記憶。
- B) 已超出模型的上下文視窗。
- C) Claude API 需要一個 `session_id` 參數。
- D) 你的應用程式沒有把先前的訊息納入 `messages` 陣列。 **[CORRECT]**

為何選 D: Claude 沒有伺服器端記憶——每次 API 呼叫都是無狀態的。若未在每個請求的 `messages` 陣列中納入完整的對話歷史,Claude 就無從得知先前的輪次。向量資料庫(A)與 `session_id`(C)都不是 Claude 架構的一部分;對於只有兩則訊息的往來而言,上下文視窗溢位(B)是不可能發生的。

問題 65(情境:對話式 AI 架構模式)

情境: 在一場 40 分鐘的烹飪工作階段後,對話達到 78,000 個 tokens。歷史內容包含過敏資訊、食譜份量縮放、已釐清的烹飪術語,以及一般討論。你必須在保留重要資訊的同時減少 tokens。

哪種做法最能在保留資訊與減少 token 之間取得平衡?

- A) 摘要整段對話歷史。
- B) 只保留最近的 20,000 個 tokens。
- C) 擷取關鍵的結構化資料(過敏資訊、數量、偏好),摘要一般討論,並逐字保留近期的往來。 **[CORRECT]**
- D) 將完整對話儲存於外部,並透過語意搜尋擷取相關部分。

為何選 C: 混合式做法以最低成本保留最高價值的資訊。像過敏資訊與食譜數量這類關鍵事實會被擷取進一個精簡的結構化區塊(避免摘要過程中發生的精確度損失),一般討論則加以摘要,而近期的往來則逐字保留以維持對話連貫性。選項 A 與 B 有遺失關鍵飲食資訊的風險;D 對於單一一場烹飪工作階段而言則是架構上的過度設計。

問題 66(情境:對話式 AI 架構模式)

情境: 使用者回報,在較長的對話中,助理會弄丟對先前主題與偏好的掌握。你目前的實作只保留最後 25 組訊息對。

最有效的解決方案為何?

- A) 混合式做法:摘要較舊的訊息,同時逐字保留近期的訊息。 **[CORRECT]**
- B) 在完整對話歷史上進行向量相似度搜尋。
- C) 將視窗增加到 50 組訊息對。
- D) 每輪都摘要被捨棄的訊息,並把持續累積的摘要前置加入。

為何選 A: 混合式做法同時處理問題的兩個面向:保留精確的近期上下文(對對話連貫性至關重要),同時維持對較早偏好的壓縮表示(避免在捨棄訊息對時造成全面遺失)。增加視窗(C)只是把同樣的問題往後延。向量搜尋(B)可能漏掉與目前查詢在語意上不相似、卻很重要的上下文。每輪完整摘要(D)會增加額外負擔,並累積摘要誤差。

問題 67(情境:對話式 AI 架構模式)

情境: 使用者回報,當對話超過 50 輪時,延遲會增加且成本會上升。

主要原因為何?

- A) 每次 API 請求都會納入整段對話歷史。 **[CORRECT]**
- B) 模型產生的回應逐漸變長。
- C) 隨著歷史增長,資料庫操作變慢。
- D) 模型建立一份內部使用者檔案,需要更多處理。

為何選 A: Claude 的 API 完全是無狀態的——每個請求都必須在 `messages` 陣列中納入完整的對話歷史。隨著對話增長,每個請求都會攜帶更多 tokens,這直接增加了處理延遲與成本。模型不會在呼叫之間維持任何

內部狀態(D 為假),而回應長度本質上也與對話長度無關(B)。

問題 68(情境:對話式 AI 架構模式)

情境: 經過三個月的每週工作階段後,對話歷史增長到 85,000 個 tokens。當使用者問「我們對於孤立這個主題做出了什麼結論?」時,助理給出的是籠統的答案,而非引用先前的討論。

最有效的做法為何?

- A) 滾動視窗截斷。
- B) 擷取關鍵結論的漸進式摘要。
- C) 以語意嵌入搭配相關往來的擷取。 **[CORRECT]**
- D) 加入結構化 XML 標籤來標記討論結論。

為何選 C: 在對話歷史上進行語意搜尋,是唯一能擴展至三個月討論量、同時又能按需求呈現特定相關往來的做法。滾動視窗(A)會捨棄大部分的歷史。漸進式摘要(B)會把討論壓縮成抽象內容,而遺失使用者正在詢問的特定結論。XML 標籤(D)需要重新編排所有過往內容,且在此規模下無法解決擷取問題。

問題 69(情境:對話式 AI 架構模式)

情境: 在 QA 測試期間,Claude 在前 10–15 輪會遵循系統提示的指引,但後續的回應卻有所偏離。對話仍在 token 上限之內。

最佳的解決方案為何?

- A) 把行為指引移到第一則使用者訊息中。
- B) 在 20 輪後開始一段新對話。
- C) 在對話的斷點處插入使用者角色的訊息以強化指引。 **[CORRECT]**
- D) 使用回應後驗證來重新產生不符合規範的回應。

為何選 C: 定期注入行為提醒,可在對話歷史累積時以固定間隔重新確立約束,直接對抗指令漂移。將指引移到第一則使用者訊息(A)會降低其權威性。開始一段新對話(B)會摧毀上下文。回應後驗證(D)屬於事後矯正而非事前預防,並會增加可觀的延遲。

問題 70(情境:對話式 AI 架構模式)

情境: 你的 AI 家教有一段 2,800 token 的系統提示,用來定義教學方法與調適規則。在 12 輪之後,助理開始忽略熟練度等級。

最有效的修正方式是什麼?

- A) 每 4–5 輪注入一次提醒。
- B) 以示範熟練度等級調適的少樣本範例取代冗長的規則。 **[CORRECT]**
- C) 將關鍵規則放在系統提示的結尾。
- D) 評估回應,若難度等級不符就重新產生。

為何選 B: 一段帶有宣告式規則的 2,800 token 系統提示容易發生漂移,因為抽象規則需要模型在每一輪都對其進行推理。以具體的少樣本範例(示範正確的熟練度等級調適)取代冗長的規則,能給模型清楚的行為模式去比對——相較於抽象指令,這種做法在多輪對話中能被更可靠地遵循。注入提醒(A)有幫助,但只處理症狀;放在結尾(C)在初期有幫助,但無法解決逐輪累積的漂移;重新產生(D)成本高昂且屬於事後補救。

問題 71(情境:對話式 AI 架構模式)

情境: 你的助理必須維持熱情的語氣、解釋其推理,並提出釐清問題。這些行為準則應該在哪裡定義?

這些行為準則應該在哪裡定義?

- A) 前置到每一則使用者訊息。
- B) 在系統提示中。 **[CORRECT]**
- C) 在第一則助理訊息中。
- D) 在環境變數中。

為何選 B: 系統提示專門設計用於整段對話中持續適用的行為約束與準則。前置到每一則使用者訊息(A)是多餘的額外負擔。第一則助理訊息(C)並不可靠,因為模型可能偏離它自己先前的陳述。環境變數(D)對模型行為沒有任何影響。

問題 72(情境:對話式 AI 架構模式)

情境: 使用者回報回應開頭重複出現像「Certainly!」和「I'd be happy to help!」這樣的句子。

最有效的做法是什麼?

- A) 附加一則帶有直接回應開頭的部分助理訊息。 **[CORRECT]**
- B) 調低 temperature 設定。
- C) 後處理回應以移除問候語。
- D) 在系統提示中加入指令以避免那些用語。

為何選 A: 以直接答覆的開頭預填助理回應,可在產生階段就防止問候語模式——模型會從預填內容接續往下,而不是產生新的開頭用語。系統提示指令(D)有幫助,但較不可靠,因為模型仍可能產生變體。後處理(C)是脆弱的權宜之計。temperature(B)控制的是隨機性,而非特定的用語模式。

問題 73(情境:對話式 AI 架構模式)

情境: 在使用者正在聊天時,一個 webhook 通知你的系統:該使用者的包裹已出貨。你希望助理在下一則回應中自然地把這項資訊納入。

最佳做法是什麼?

- A) 將出貨狀態加入系統提示。
- B) 立即送出一則合成的使用者訊息。
- C) 強制助理在每一輪都呼叫一個狀態工具。

- D) 將狀態更新附加為下一則使用者訊息的前綴。 **[CORRECT]**

為何選 D: 把狀態更新前綴到下一則使用者訊息,可在自然的對話邊界注入即時上下文,而不會打斷流程。修改系統提示(A)需要重建工作階段,或在架構上很麻煩。合成的使用者訊息(B)可能破壞自然的對話流程並混淆出處標註。強制每一輪都呼叫工具(C)在事件罕見時很浪費。

問題 74(情境:對話式 AI 架構模式)

情境: 使用者經常送出像「幫派對訂一個場地」這樣的請求。助理會問 4 個以上的釐清問題,導致 35% 的放棄率。

哪種做法最能改善這個權衡取捨?

- A) 以隱藏的預設值逕行進行。
- B) 在一則複合訊息中問完所有釐清問題。
- C) 明確陳述假設並逕行進行,同時邀請使用者更正。 **[CORRECT]**
- D) 使用結構化的填寫表單。

為何選 C: 明確陳述假設並逕行進行,能立即給使用者一個有用的回應,同時保留他們更正錯誤假設的能力。隱藏的預設值(A)讓使用者不知道系統假設了什麼。複合式的問題清單(B)仍要求使用者預先付出心力。結構化表單(D)增加而非減少摩擦——與降低放棄率的目標相矛盾。

問題 75(情境:對話式 AI 架構模式)

情境: 你的助理使用一段承包商人設的系統提示。前幾輪都遵循規則,但到了第 7 輪,助理開始給出籠統的建議。對話長度只有 2,500 token。

最可能的原因是什麼?

- A) 系統提示只建立初始行為。
- B) 隨著輪數累積,模型注意力減弱。
- C) 累積的助理回應稀釋了系統提示的影響力。 **[CORRECT]**
- D) 系統提示只送出一次。

為何選 C: 隨著助理回應在對話歷史中累積,反映系統提示行為約束的文字比例,相對於不斷增長的助理產生內容而下降。模型會越來越傾向比對自己先前的輸出,而非系統提示,即使在較短的 token 長度下也會加劇漂移。系統提示包含在每一次 API 呼叫中(就單獨解釋而言,D 是錯的),而模型注意力衰退(B)在 2,500 token 下並不會發生。

問題 76(情境:對話式 AI 架構模式)

情境: 使用者提出像「你能幫忙處理那份報告嗎?」這樣模糊的請求。助理以提出多個問題作為回應(哪份報告?需要什麼幫助?截止期限?),導致 40% 的放棄率。

最佳解決方案是什麼?

- A) 做出合理的假設、明確陳述,並提議進行調整。 **[CORRECT]**
- B) 在回應前用較小的模型對模糊性進行分類。
- C) 使用預先定義的詮釋,但不陳述假設。
- D) 限制助理每輪只能問一個釐清問題。

為何選 A: 以合理且明說的假設逕行進行,可完全消除來回往返,同時讓使用者知情並保有掌控。預先定義且不明說的詮釋(C)在回應不符合使用者意圖時讓他們困惑。每輪一個問題的限制(D)仍需要多輪來回往返。較小的分類模型(B)增加延遲與基礎架構複雜度,卻沒有解決核心的使用者體驗問題。

實作練習

練習 1:具升級邏輯的多工具代理

目標: 設計一個整合工具、具結構化錯誤處理與升級機制的代理迴圈。

步驟:

1. 定義 3–4 個 MCP 工具並附上詳細描述(包含兩個相似的工具以測試工具選擇)
2. 實作一個檢查 `stop_reason`(`"tool_use"` / `"end_turn"`)的代理迴圈
3. 加入結構化的錯誤回應:`errorCategory`、`isRetryable`、描述
4. 實作一個攔截器 hook,封鎖超過門檻的操作並轉送至升級流程
5. 以多面向的請求進行測試

涵蓋領域: 1(代理架構)、2(工具與 MCP)、5(上下文與可靠性)

練習 2:為團隊開發設定 Claude Code

目標: 設定 CLAUDE.md、自訂命令、特定路徑規則,以及 MCP 伺服器。

步驟:

1. 建立一個含通用標準的專案層級 CLAUDE.md
2. 在 `.claude/rules/` 建立帶有 YAML frontmatter 的檔案,對應不同的程式碼範圍(`paths: ["src/api/**/*"]`、`paths: ["**/*.test.*"]`)
3. 在 `.claude/skills/` 下建立一個專案技能,設定 `context: fork` 與 `allowed-tools`
4. 在 `.mcp.json` 中設定一個帶環境變數的 MCP 伺服器,並在 `~/.claude.json` 中加上個人覆寫設定
5. 在不同複雜度的任務上測試規劃模式與直接執行的差異

涵蓋領域: 3(Claude Code 設定)、2(工具與 MCP)

練習 3:結構化資料擷取管線

目標: JSON 結構描述、用於結構化輸出的 `tool_use`、驗證／重試迴圈、批次處理。

步驟:

1. 定義一個帶 JSON 結構描述的擷取工具(必填／選填欄位、含 "other" 的列舉、可為 null 的欄位)
2. 建立一個驗證迴圈:發生錯誤時,帶上文件、錯誤的擷取結果,以及具體的驗證錯誤進行重試
3. 為不同結構的文件加入少樣本範例
4. 透過 Message Batches API 進行批次處理:100 份文件,藉由 `custom_id` 處理失敗的項目
5. 轉送至真人處理:欄位層級的信心分數、文件類型分析

涵蓋領域: 4(提示工程)、5(上下文與可靠性)

練習 4:設計與除錯多代理研究管線

目標: 子代理編排、上下文傳遞、錯誤傳播,以及附帶來源追蹤的彙整。

步驟:

1. 一個協調者搭配 2 個以上的子代理(`allowedTools` 包含 `"Task"`,上下文在提示中明確傳遞)
2. 在單一回應中透過多個 `Task` 呼叫平行執行子代理
3. 要求子代理產出結構化輸出:主張、引述、來源 URL、發布日期
4. 模擬子代理逾時:將結構化的錯誤上下文回傳給協調者,並以部分結果繼續執行
5. 以衝突資料進行測試:保留兩個數值並標註出處;將已確認與有爭議的發現分開

涵蓋領域: 1(代理架構)、2(工具與 MCP)、5(上下文與可靠性)

附錄:技術與概念

技術	重點面向
Claude Agent SDK	AgentDefinition、代理迴圈、`stop_reason`、hooks(PostToolUse)、透過 Task 生成子代理、`allowedTools`
模型上下文協定(MCP)	MCP 伺服器、工具、資源、`isError`、工具描述、`.mcp.json`、環境變數
Claude Code	CLAUDE.md 階層、帶 glob 模式的 `.claude/rules/`、`.claude/commands/`、含 SKILL.md 的 `.claude/skills/`、規劃模式、`/compact`、`--resume`、`fork_session`
Claude Code CLI	用於非互動模式的 `-p` / `--print`、`--output-format json`、`--json-schema`
Claude API	帶 JSON 結構描述的 `tool_use`、`tool_choice`（"auto"／"any"／強制）、`stop_reason`、`max_tokens`、系統提示
Message Batches API	節省 50%、最長 24 小時視窗、`custom_id`、不支援多輪工具呼叫

JSON Schema	必填與選填、可為 null 的欄位、列舉型別、"other" + 細節、嚴格模式
Pydantic	結構描述驗證、語意錯誤、驗證／重試迴圈
內建工具	Read、Write、Edit、Bash、Grep、Glob — 用途與選用標準
少樣本提示	針對不明確情境的目標式範例、對新模式的泛化
提示鏈接	循序分解為聚焦的多個階段
上下文視窗	token 預算、漸進式摘要、「迷失在中間」、暫存檔(scratchpad)
工作階段管理	恢復、`fork_session`、具名工作階段、上下文隔離
信心校準	欄位層級評分、在已標註資料集上的校準、分層抽樣

範圍外主題

以下相鄰主題**不會**出現在考試中:

- 微調 Claude 模型或訓練自訂模型
- Claude API 驗證、計費或帳戶管理
- 在特定程式語言或框架中的詳細實作(超出工具／結構描述設定所需的範圍)
- 部署或託管 MCP 伺服器(基礎架構、網路、容器編排)
- Claude 的內部架構、訓練程序或模型權重
- Constitutional AI、RLHF 或安全訓練方法論
- 嵌入模型或向量資料庫的實作細節
- 電腦使用(瀏覽器自動化、桌面互動)
- 影像分析能力(Vision)
- 串流 API 或 server-sent events
- 速率限制、配額或詳細的 API 成本計算
- OAuth、API 金鑰輪替或驗證協定細節
- 雲端供應商專屬設定(AWS、GCP、Azure)
- 效能基準測試或模型比較指標
- 提示快取的實作細節(僅需知道其存在)
- token 計數演算法或 tokenization 細節

準備建議

1. **使用 Claude Agent SDK 建構一個代理** — 實作一個完整的代理迴圈,包含工具呼叫、錯誤處理與工作階段管理。練習子代理與明確的上下文傳遞。

2. **為真實專案設定 Claude Code** — 運用 CLAUDE.md 階層、`.claude/rules/` 中的路徑專屬規則、搭配 `context: fork` 與 `allowed-tools` 的技能,以及 MCP 伺服器整合。
3. **設計並測試 MCP 工具** — 撰寫能區分相似工具的描述、回傳帶有類別與重試旗標的結構化錯誤,並針對不明確的使用者請求進行測試。
4. **建構一個資料擷取管線** — 使用搭配 JSON 結構描述的 `tool_use`、驗證/重試迴圈、可選/可為 null 的欄位,以及透過 Message Batches API 的批次處理。
5. **練習提示工程** — 為不明確的情境加入少樣本範例、明確的審查標準,以及用於大型程式碼審查的多遍架構。
6. **研讀上下文管理模式** — 從冗長的輸出中擷取事實、使用暫存檔(scratchpad),並將探查工作委派給子代理以因應上下文限制。
7. **理解升級與人在迴路(human-in-the-loop)** — 何時該升級(政策缺口、使用者明確要求、無法取得進展)以及以信心為基礎的路由工作流程。
8. 在正式考試前先**做一份模擬考**。它使用相同的情境與格式。